



Decision Making Under Uncertainty with Markov Decision Processes

Liam Kruse

Lecture 05 | AE 513 Multivariable Controls



Liam Kruse

liam.a.kruse@gmail.com



The Boeing Company

AI Scientist, Caravan Team



Stanford University

MS, PhD in Aeronautics and Astronautics



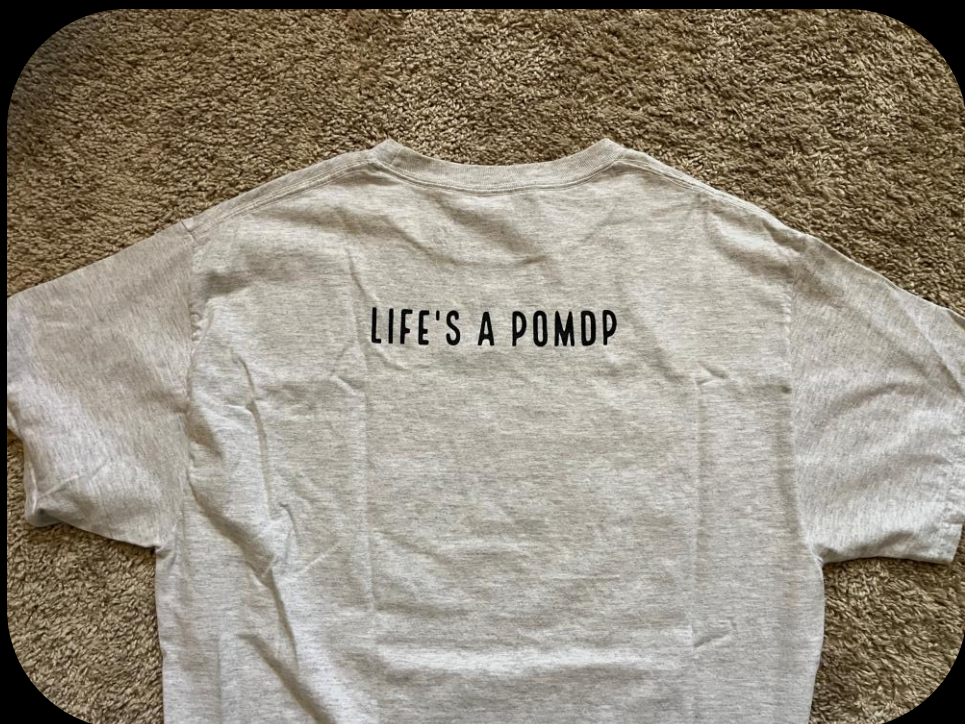
Stanford Intelligent Systems Lab

Safe AI and decision-making under uncertainty



University of Nebraska–Lincoln

BS in Mechanical Engineering



Last Time

- **System** $x_{t+1} = f(x_t, u_t, t), t = 0, \dots, N - 1$, **control constraints** $u_t \in U(x_t)$.
- **Cost** $J(x_0, u_{0:N-1}) = g_N(x_N) + \sum_{t=0}^{N-1} g(x_t, u_t, t)$

Last Time

- Given an optimal policy $\mathbf{u}_t^* = \pi^*(\mathbf{x}_t, t)$, starting at $\mathbf{x}_0$
- Optimal cost is $J^*(\mathbf{x}_0, 0) = g_N(\mathbf{x}_N) + \sum_{t=0}^{N-1} g(\mathbf{x}_t, \pi^*(\mathbf{x}_t, t), t)$
- At $(\mathbf{x}_t, t)$ the optimal cost is

$$J^*(\mathbf{x}_t, t) = g_N(\mathbf{x}_N) + \sum_{k=t}^{N-1} g(\mathbf{x}_k, \pi^*(\mathbf{x}_k, k), k)$$

- Alternatively,

$$J^*(\mathbf{x}_t, t) = g(\mathbf{x}_t, t) + g_N(\mathbf{x}_N) + \sum_{k=t+1}^{N-1} g(\mathbf{x}_k, \pi^*(\mathbf{x}_k, k), k)$$

$$J^*(\mathbf{x}_t, t) = g(\mathbf{x}_t, t) + J^*(\mathbf{x}_{t+1}, t+1)$$

Last Time

- Cost-to-go, also referred to as the **Value Function**
 - The future total cost at $(\mathbf{x}_t, t)$ if following policy π

$$V_{\pi}(\mathbf{x}_t, t) = g_N(\mathbf{x}_N) + \sum_{k=t}^{N-1} g(\mathbf{x}_k, \pi(\mathbf{x}_k, k), k)$$

$$V_{\pi}(\mathbf{x}_t, t) = g(\mathbf{x}_t, \pi(\mathbf{x}_t, t), t) + V_{\pi}(\mathbf{x}_{t+1}, t + 1)$$

- Optimal value function corresponds to following optimal policy

Last Time

$$V^*(\mathbf{x}_t, t) = \min_{u_t \in U} g(\mathbf{x}_t, u_t, t) + V^*(f(\mathbf{x}_t, u_t, t), t + 1)$$

$$\pi^*(\mathbf{x}_t, t) = \operatorname{argmin}_{u_t \in U} g(\mathbf{x}_t, u_t, t) + V^*(f(\mathbf{x}_t, u_t, t), t + 1)$$

- What about continuous state and control (still discrete time)?
- What about stochasticity?
- What about infinite horizon?

Terminology

States

“The minimum set of information required to make a good decision.”

s instead of x



$s = (x, y, v, \text{health})$

Actions

The inputs that an agent takes to influence their environment

a instead of u

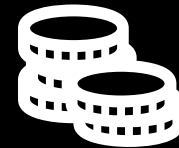


$a = \{\leftarrow, \uparrow, \rightarrow, \downarrow\}, \{\ddot{a}_1, \ddot{a}_2, \ddot{a}_3\}$

Rewards

A deterministic function of state and action; + for *reward*, - for *penalty*

$R(s, a)$ instead of $g(x, u)$



Terminology

Utility

The (discounted) sum of expected rewards from starting in state s and following policy π

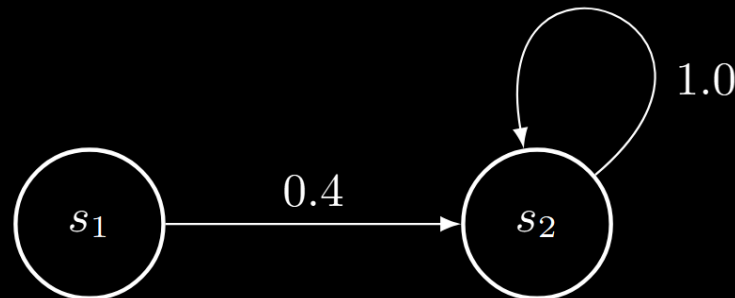
$U^\pi(s), U^*(s)$ instead of $V^\pi(x), V^*(x)$ for *value function*

Stochastic Transition Model

We are interested in *decision making under uncertainty*; what if we have *outcome uncertainty* due to stochastic dynamics?

Examples: UAV flight in wind, biological response to medicine

$T(s' | s, a) = \Pr(s' | s, a)$ instead of $f(x, u)$



	s_1	s_2
s_1	0.6	0.4
s_2	0.0	1.0

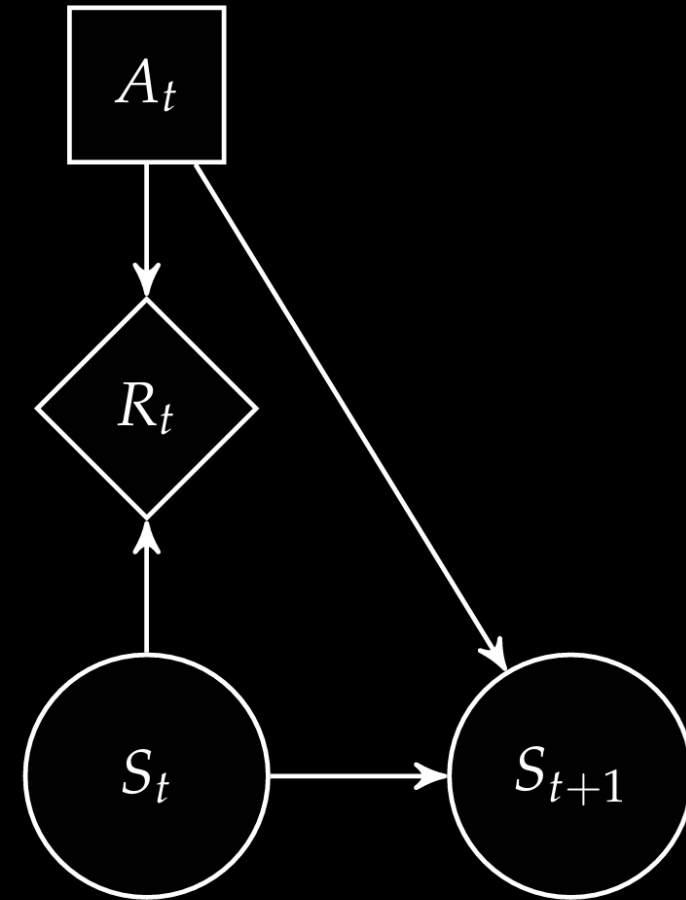
Markov Decision Process

State Space $\mathcal{S}$

Action Space $\mathcal{A}$

Transition Model $T(s' | s, a)$

Reward Function $R(s, a)$



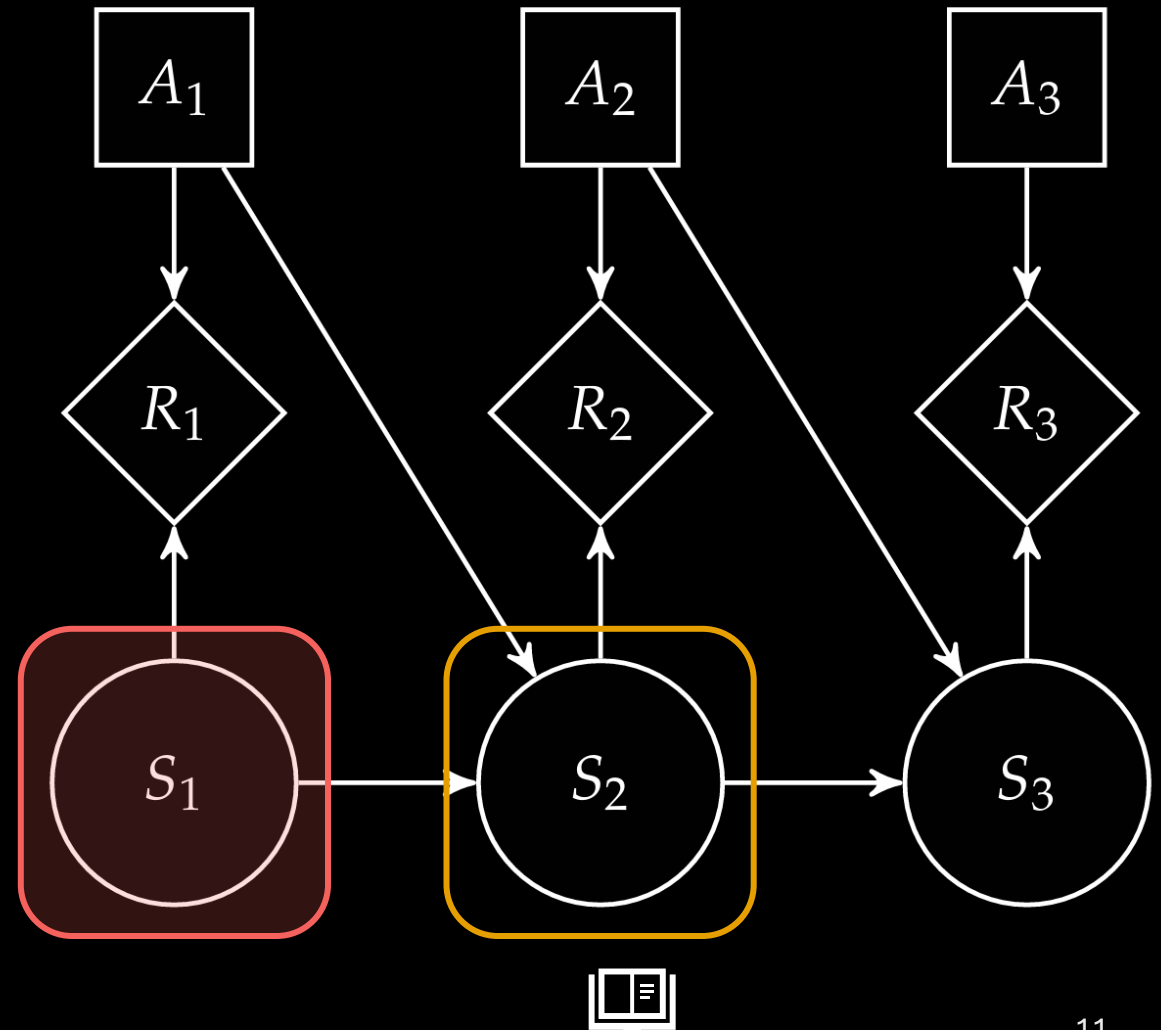
Markov Decision Process

State Space $\mathcal{S}$

Action Space $\mathcal{A}$

Transition Model $T(s' | s, a)$

Reward Function $R(s, a)$



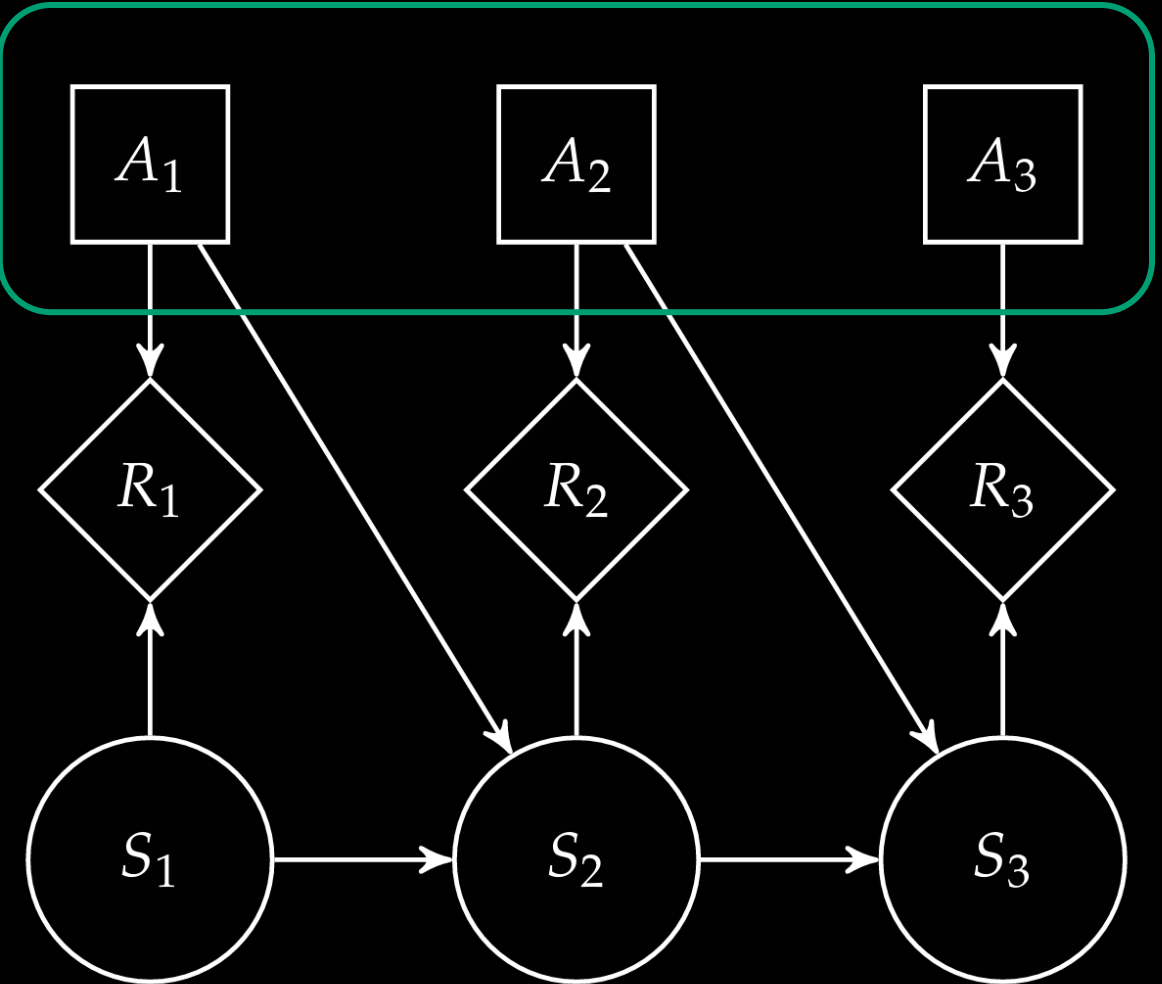
Markov Decision Process

State Space $\mathcal{S}$

Action Space $\mathcal{A}$

Transition Model $T(s' | s, a)$

Reward Function $R(s, a)$



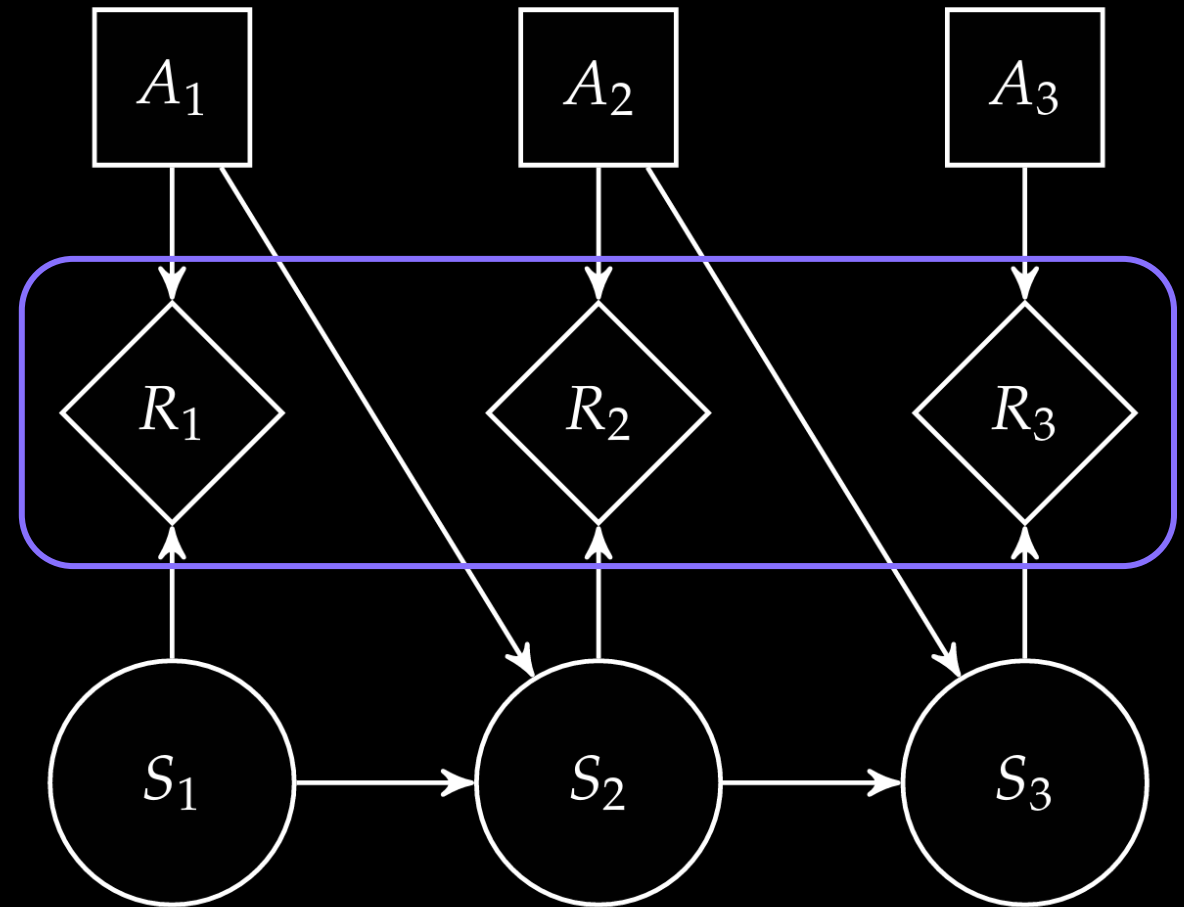
Markov Decision Process

State Space $\mathcal{S}$

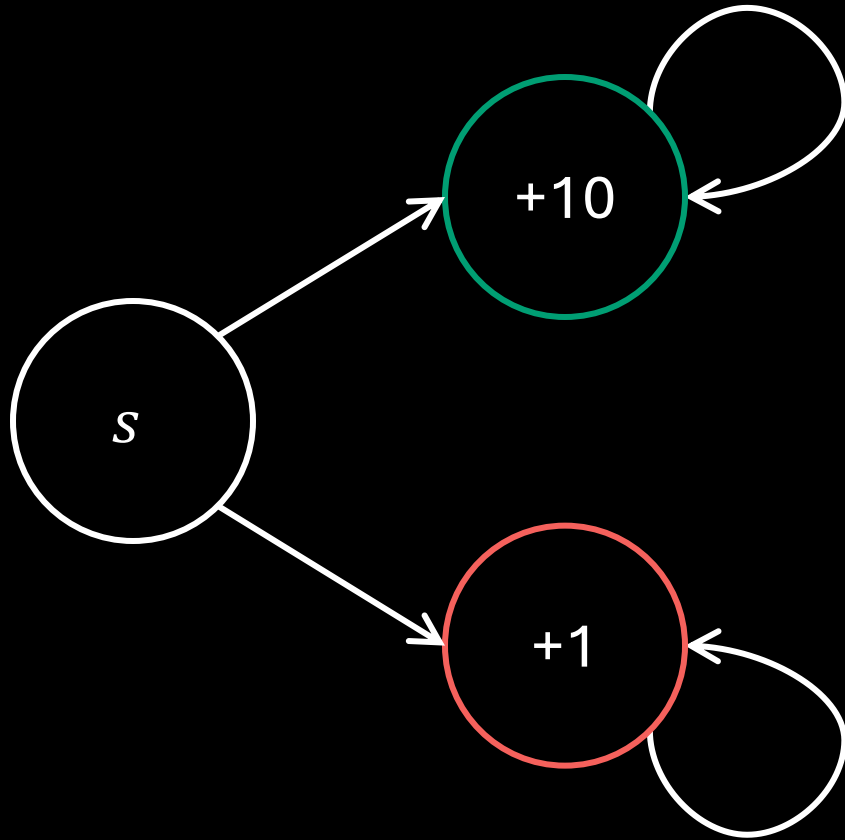
Action Space $\mathcal{A}$

Transition Model $T(s' | s, a)$

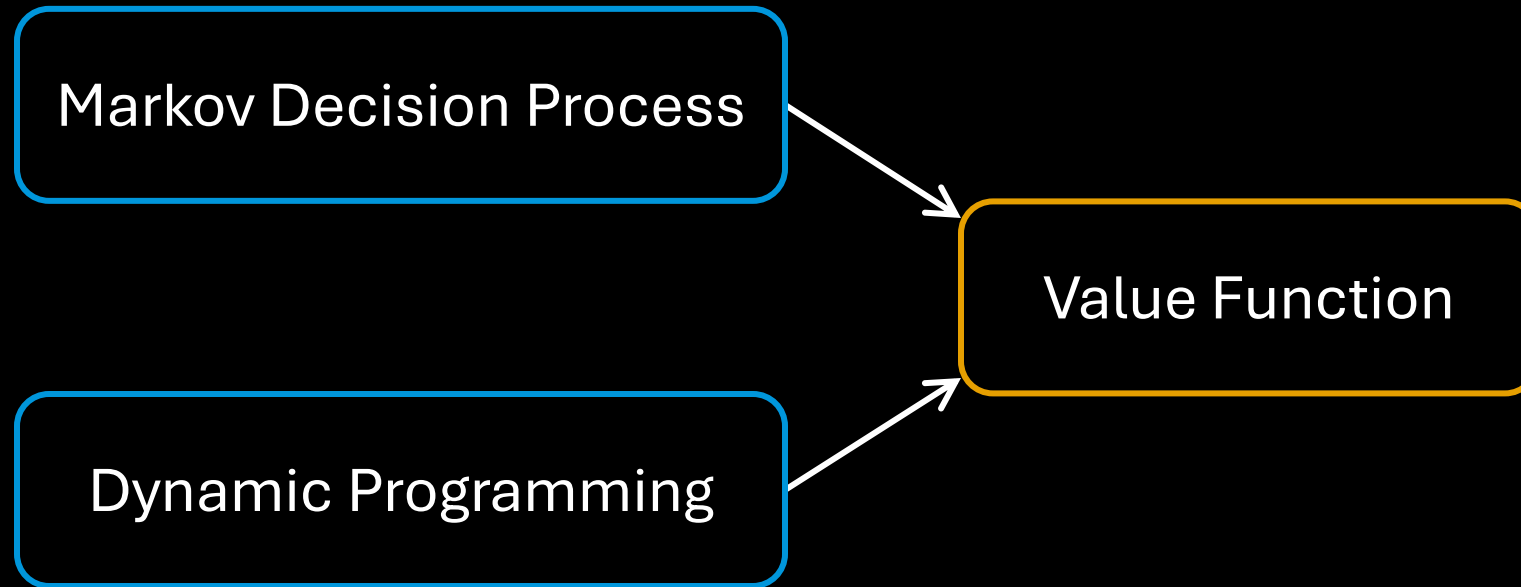
Reward Function $R(s, a)$



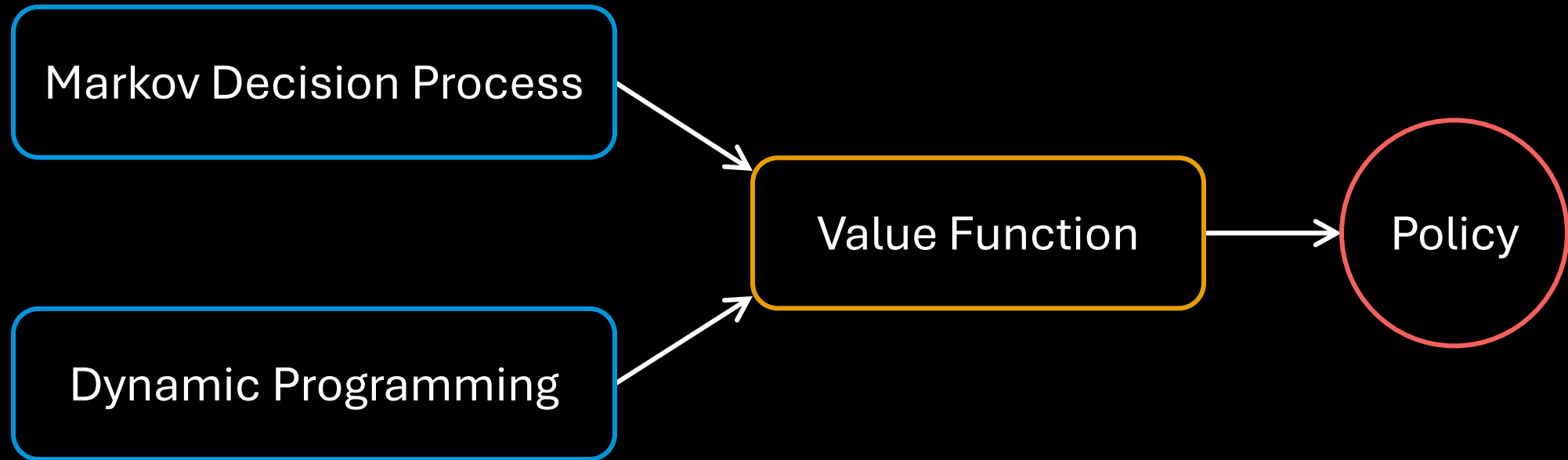
Discounting



Markov Decision Process



Markov Decision Process



			+10
			-5

			+10
			-5

→	→	→	+10
↑		↑	-5
↑	←	↑	←

The Bellman Equation

$$U_{k+1}(s) = \max_a \left(R(s, a) + \gamma \sum_{s'} T(s' \mid s, a) U_k(s') \right)$$

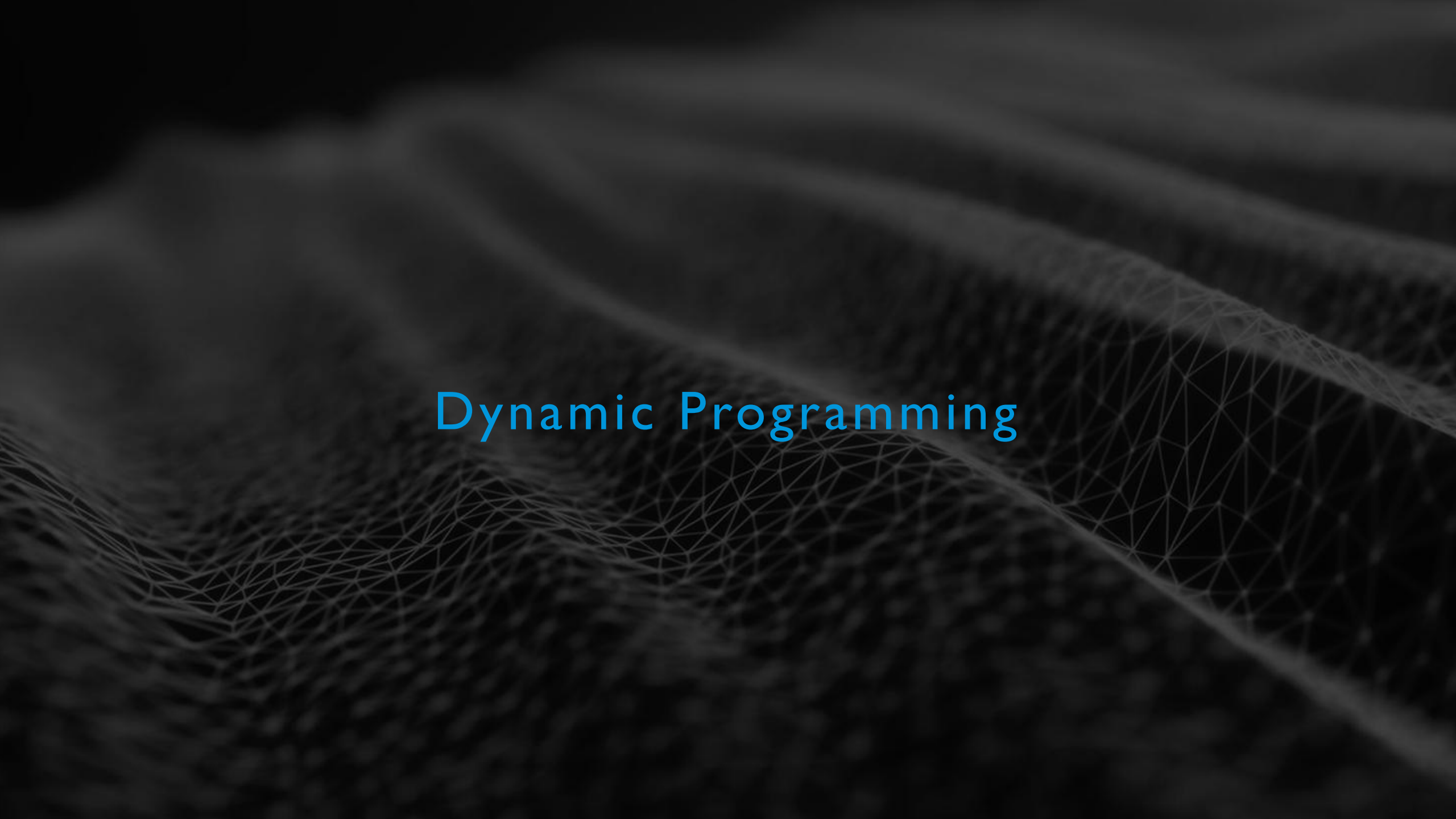
The Bellman Equation

Last week's formulation:

$$V^*(\mathbf{x}_t, t) = \min_{u_t \in U} g(\mathbf{x}_t, u_t, t) + \underline{V^*(f(\mathbf{x}_t, u_t, t), t + 1)}$$

Today:

$$U_{k+1}(s) = \max_a \left(R(s, a) + \gamma \sum_{s'} T(s' \mid s, a) U_k(s') \right)$$

The background is a dark, monochromatic abstract image. It features a complex, wavy pattern that resembles a mesh or a series of overlapping, translucent layers. The lines are thin and light gray, creating a sense of depth and movement against the dark background. The overall effect is futuristic and technical.

Dynamic Programming



Value Iteration

Learn an optimal value function, then extract an optimal policy



Asynchronous Value Iteration

Value iteration with some tricks to speed up convergence



Policy Iteration

Alternate between policy evaluation and policy improvement

Value Iteration

Algorithm 1 Infinite Horizon Value Iteration

Require: discount factor γ , maximum iterations $k_{\max}$

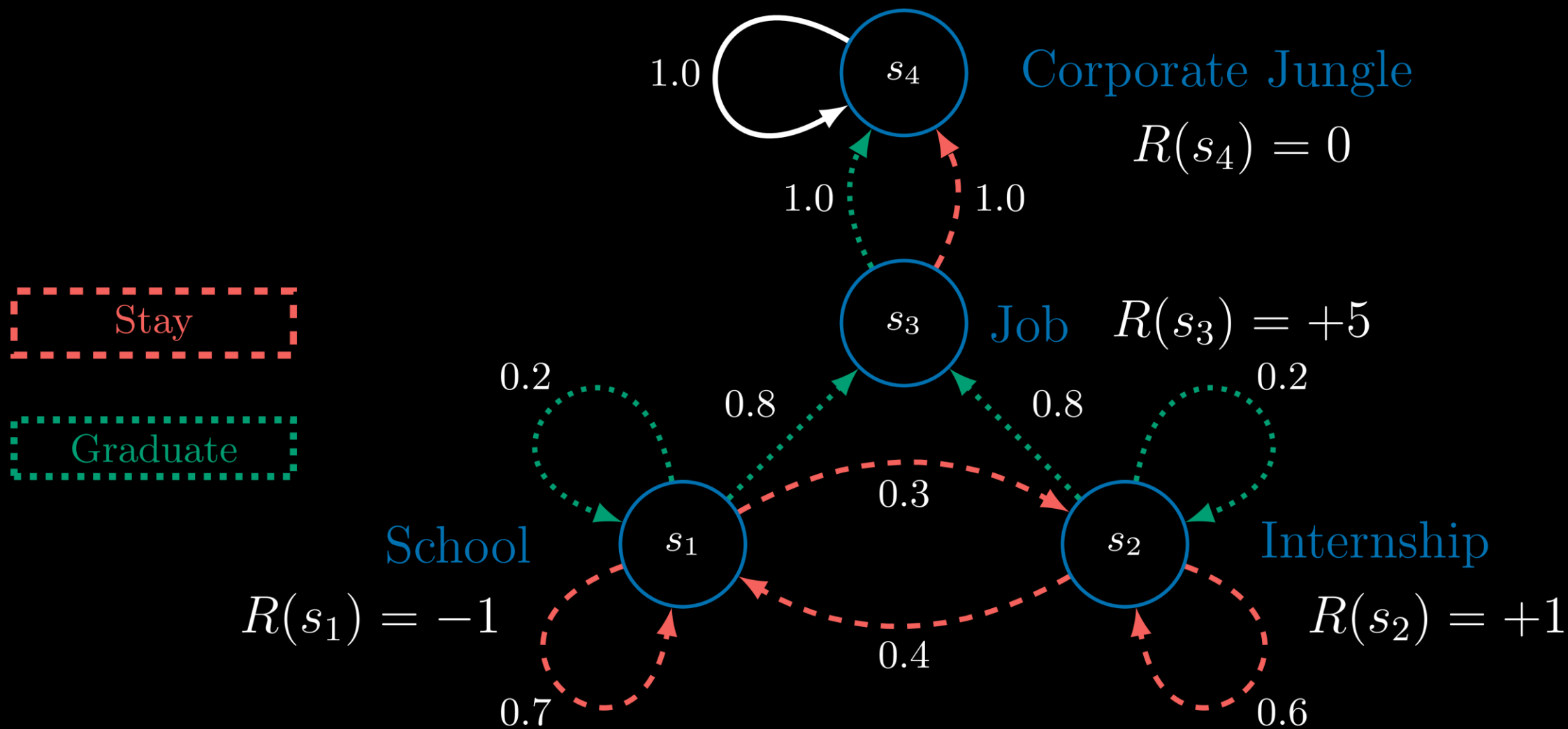
1: $U(s) \leftarrow 0$ for all $s \in \mathcal{S}$

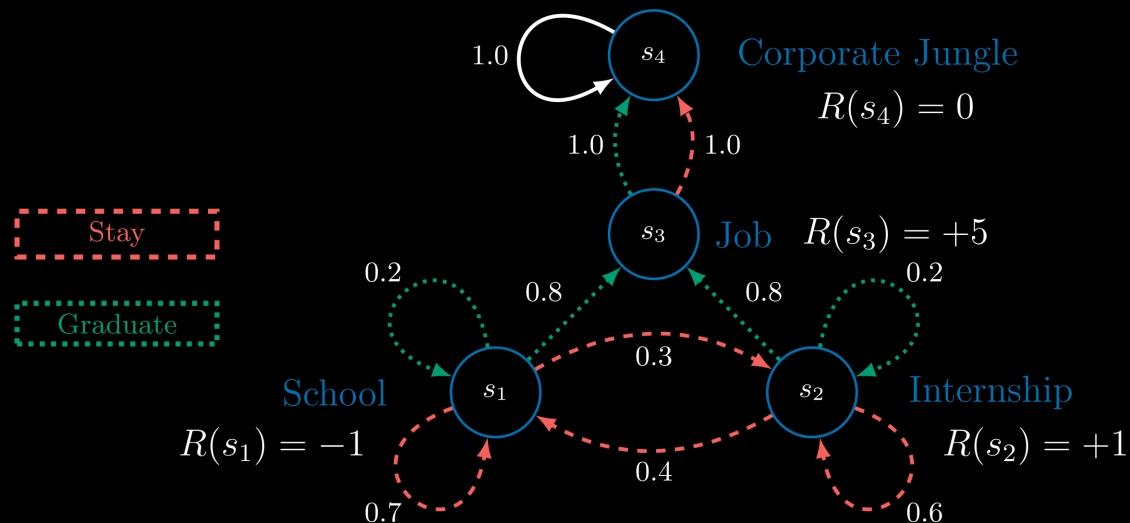
2: **for** $k \leftarrow 1, k_{\max}$ **do**

3: **for** each $s \in \mathcal{S}$ **do**

4: $U_{k+1}(s) = \max_a (R(s, a) + \gamma \sum_{s'} T(s' | s, a) U_k(s'))$

5: **return** $\pi(s) = \operatorname{argmax}_a (R(s, a) + \gamma \sum_{s'} T(s' | s, a) U^*(s'))$ for each $s \in \mathcal{S}$

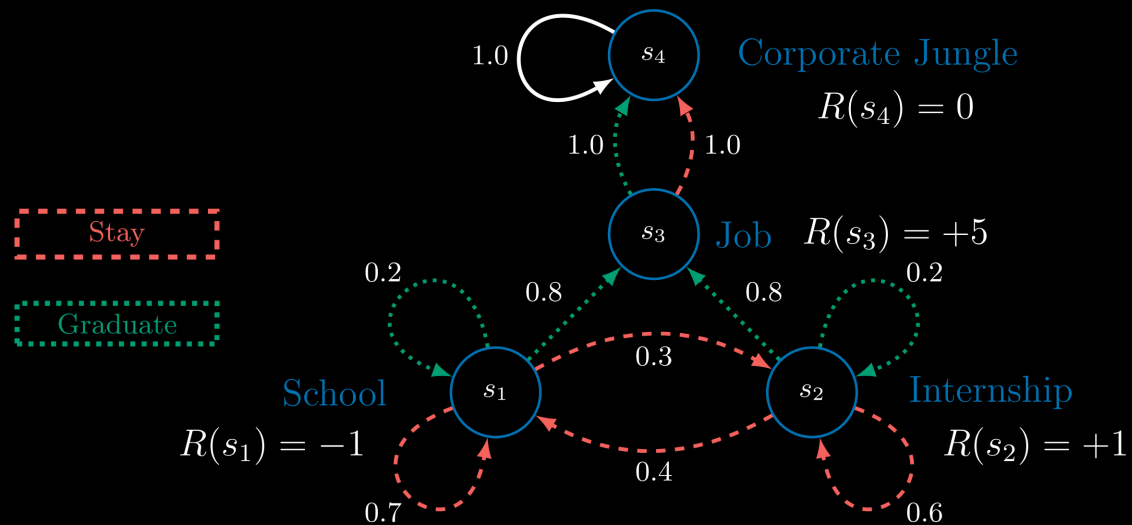




Iteration 1

$$U_1(s_1) = -1 + 0.9 \max_a \left\{ \underbrace{0.7 \times 0 + 0.3 \times 0}_{\text{Stay: } 0.0}, \underbrace{0.2 \times 0 + 0.8 \times 0}_{\text{Grad: } 0.0} \right\} = -1$$

$$U_{k+1}(s) = \max_a \left(R(s, a) + \gamma \sum_{s'} T(s' | s, a) U_k(s') \right)$$



Iteration 2

$$U_2(s_1) = -1 + 0.9 \max_a \left\{ \underbrace{0.7 \times -1 + 0.3 \times 1}_{\text{Stay: } -0.4}, \underbrace{0.8 \times 5 + 0.2 \times -1}_{\text{Grad: } 3.8} \right\} = 2.42$$

Asynchronous Value Iteration

Algorithm 2 Gauss Seidel Value Iteration

Require: discount factor γ , maximum iterations $k_{\max}$

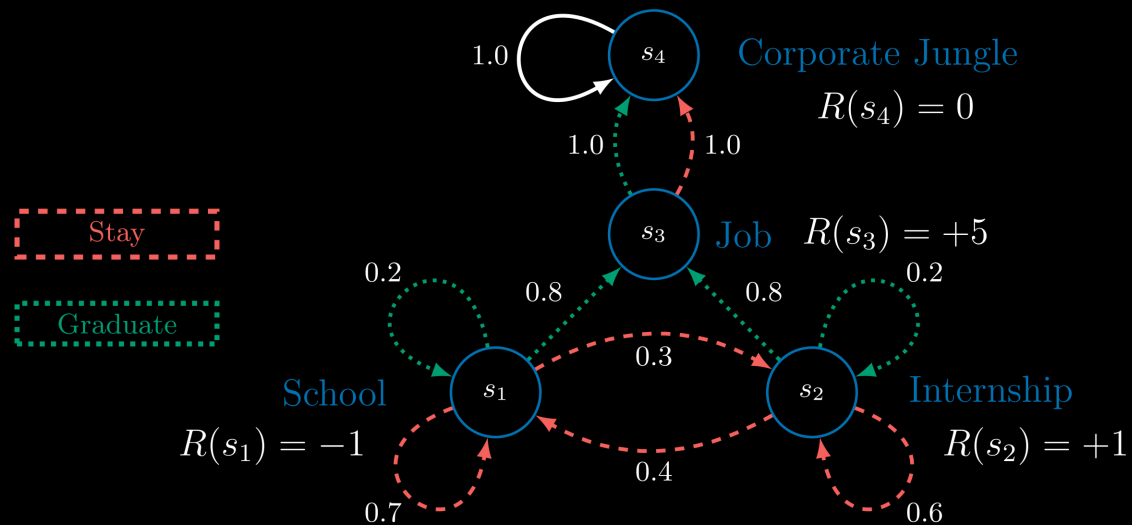
1: $U(s) \leftarrow 0$ for all $s \in \mathcal{S}$

2: **for** $k \leftarrow 1, k_{\max}$ **do**

3: **for** each $s \in \mathcal{S}$ **do**

4: $U_{k+1}(s) = \max_a (R(s, a) + \gamma \sum_{s'} T(s' | s, a) \cancel{U_k(s')} U_{k+1}(s'))$

5: **return** $\pi(s) = \operatorname{argmax}_a (R(s, a) + \gamma \sum_{s'} T(s' | s, a) U^*(s'))$ for each $s \in \mathcal{S}$



Iteration 1

$$U_1(s_1) = -1 + 0.9 \max_a \{ \underbrace{0.7 \times 0 + 0.3 \times 0}_{\text{Stay: } 0.0}, \underbrace{0.2 \times 0 + 0.8 \times 0}_{\text{Grad: } 0.0} \} = -1$$

Policy Iteration (part I)

Algorithm 3 Infinite Horizon Policy Evaluation

Require: policy π , discount factor γ , maximum iterations $k_{\max}$

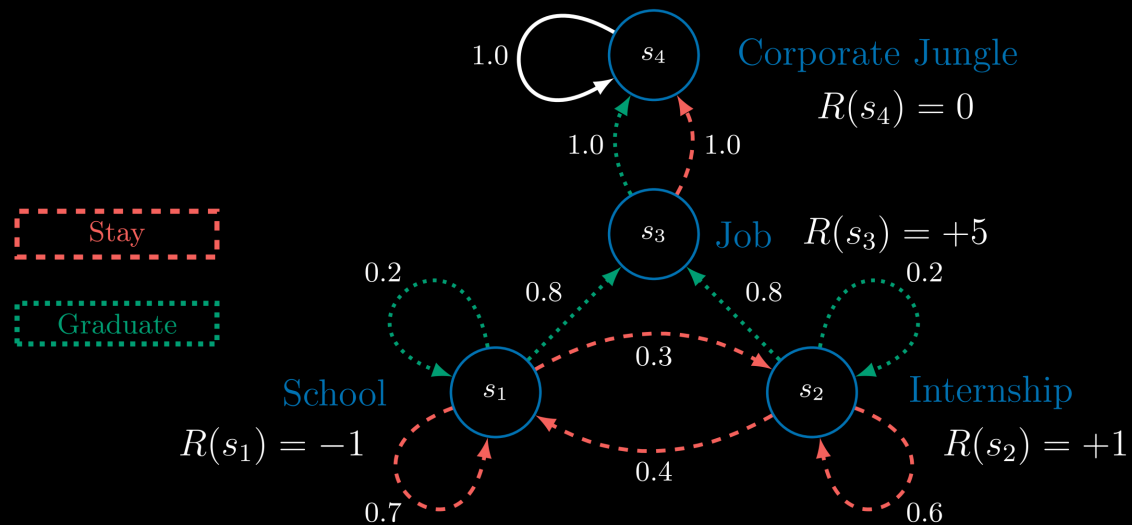
- 1: $U(s) \leftarrow 0$ for all $s \in \mathcal{S}$
- 2: **for** $k \leftarrow 1, k_{\max}$ **do**
- 3: **for** each $s \in \mathcal{S}$ **do**
- 4: $U_{k+1}(s) = R(s, \pi(s)) + \gamma \sum_{s'} T(s' \mid s, \pi(s)) U_k(s')$
- 5: **return** $U(s)$ for each $s \in \mathcal{S}$

Policy Iteration (part 2)

Algorithm 4 Infinite Horizon Policy Iteration

Require: discount factor γ

- 1: $U(s) \leftarrow 0$ for all $s \in \mathcal{S}$
- 2: initialize $\pi(s), \hat{\pi}(s)$ for all s arbitrarily, with $\pi(s) \neq \hat{\pi}(s)$ for some $s \in \mathcal{S}$
- 3: **while** $\pi(s) \neq \hat{\pi}(s)$ for some $s \in \mathcal{S}$ **do**
- 4: $\pi(s) \leftarrow \hat{\pi}(s)$ for all $s \in \mathcal{S}$
- 5: $U \leftarrow$ Policy Evaluation with policy π
- 6: $\hat{\pi}(s) \leftarrow \operatorname{argmax}_a (R(s, a) + \gamma \sum_{s'} T(s' | s, a) U(s'))$ for each $s \in \mathcal{S}$
- 7: **return** $\pi(s) = \operatorname{argmax}_a (R(s, a) + \gamma \sum_{s'} T(s' | s, a) U(s'))$ for each $s \in \mathcal{S}$

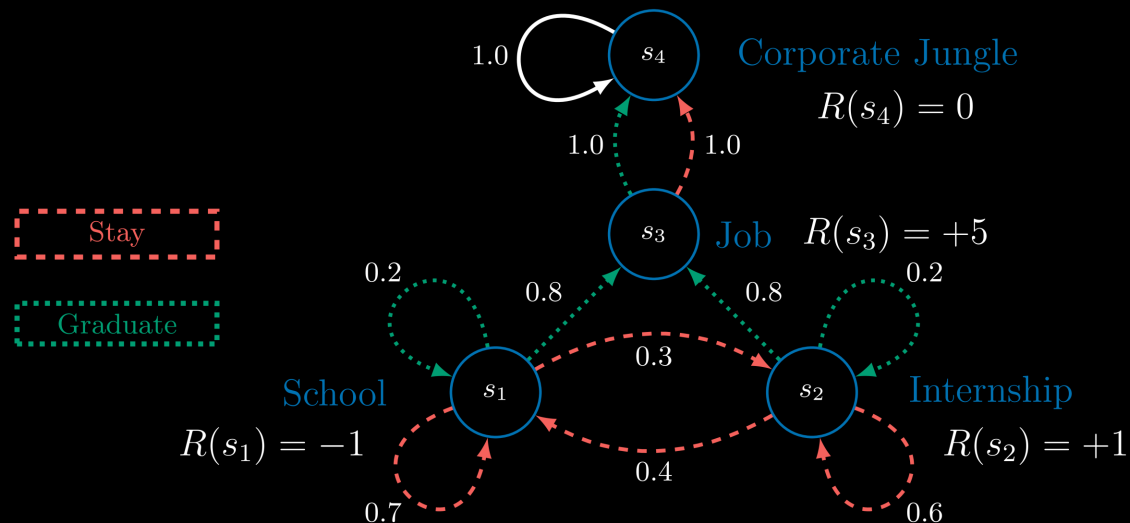


Policy Evaluation

$$U_1^\pi(s_1) = -1 + 0.9 \underbrace{\{0.7 \times 0 + 0.3 \times 0\}}_{\text{Stay: } 0.0} = -1$$

$$U_1^\pi(s_2) = +1 + 0.9 \underbrace{\{0.6 \times 0 + 0.4 \times 0\}}_{\text{Stay: } 0.0} = 1$$

$$U_1^\pi(s_3) = +5 + 0.9 \underbrace{\{0\}}_{\text{Stay: } 0.0} = +5$$

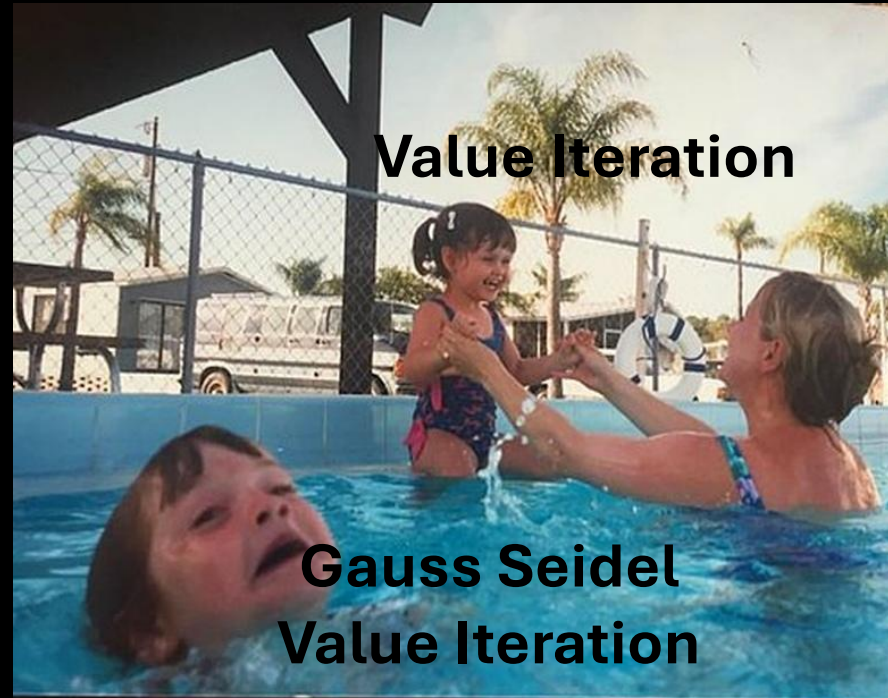


Policy Improvement

$$\pi(s_1) = \arg \max_a \{-1 + 0.9 \underbrace{\{0.7 \times -1 + 0.3 \times 1\}}_{\text{Stay: } -0.4}, \underbrace{\{0.8 \times 5 + 0.2 \times -1\}}_{\text{Grad: } 3.8}\} = \text{Grad}$$

$$\pi(s_2) = \arg \max_a \{+1 + 0.9 \underbrace{\{0.6 \times 1 + 0.4 \times -1\}}_{\text{Stay: } 0.2}, \underbrace{\{0.8 \times 5 + 0.2 \times 1\}}_{\text{Grad: } 4.2}\} = \text{Grad}$$

$$\pi(s_3) = \arg \max_a \{+5 + 0.9 \{ \underbrace{0}_{\text{Stay: } 0.0}, \underbrace{0}_{\text{Grad: } 0.0} \} \} = \text{Grad, Stay}$$

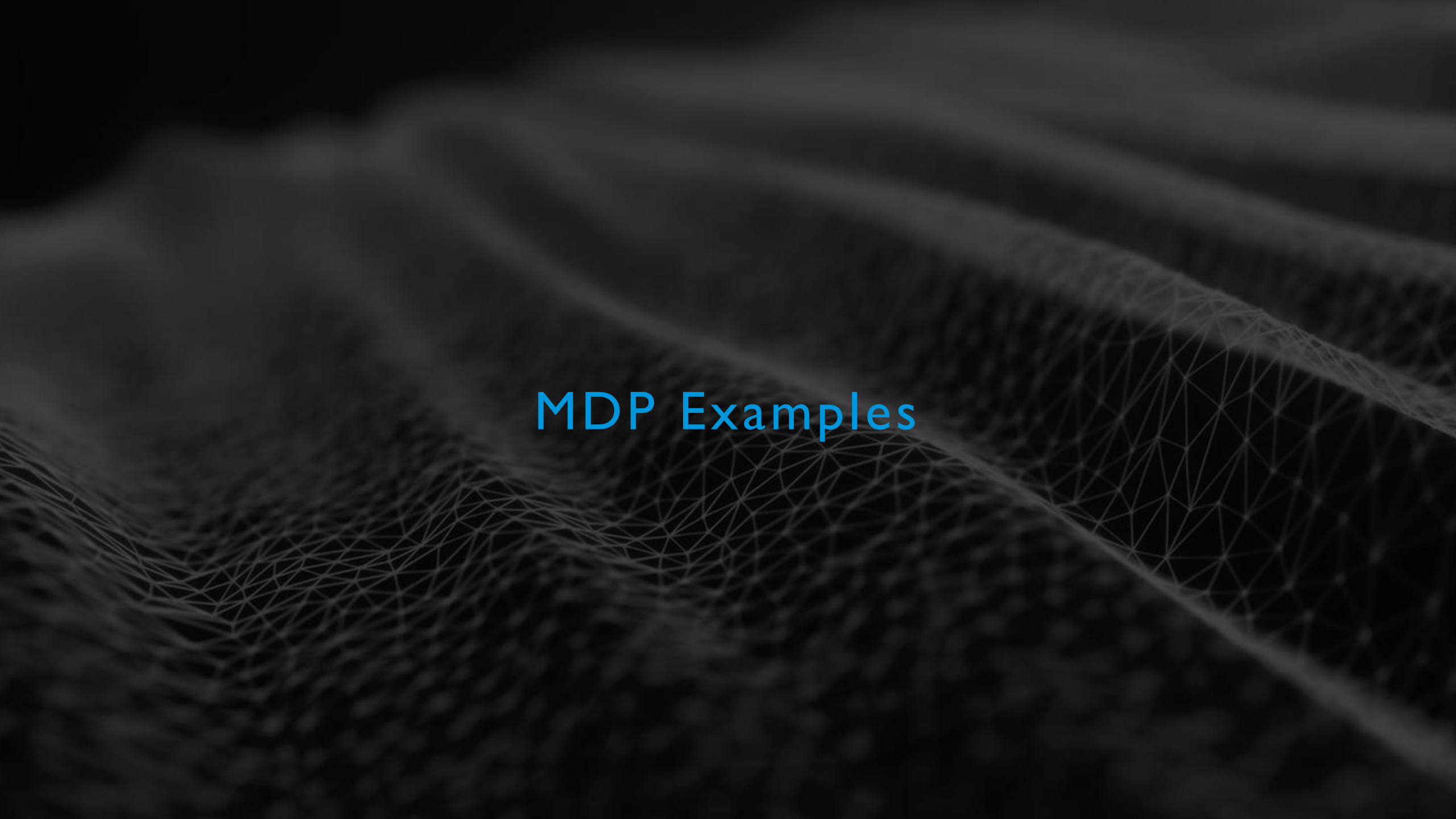


Value Iteration

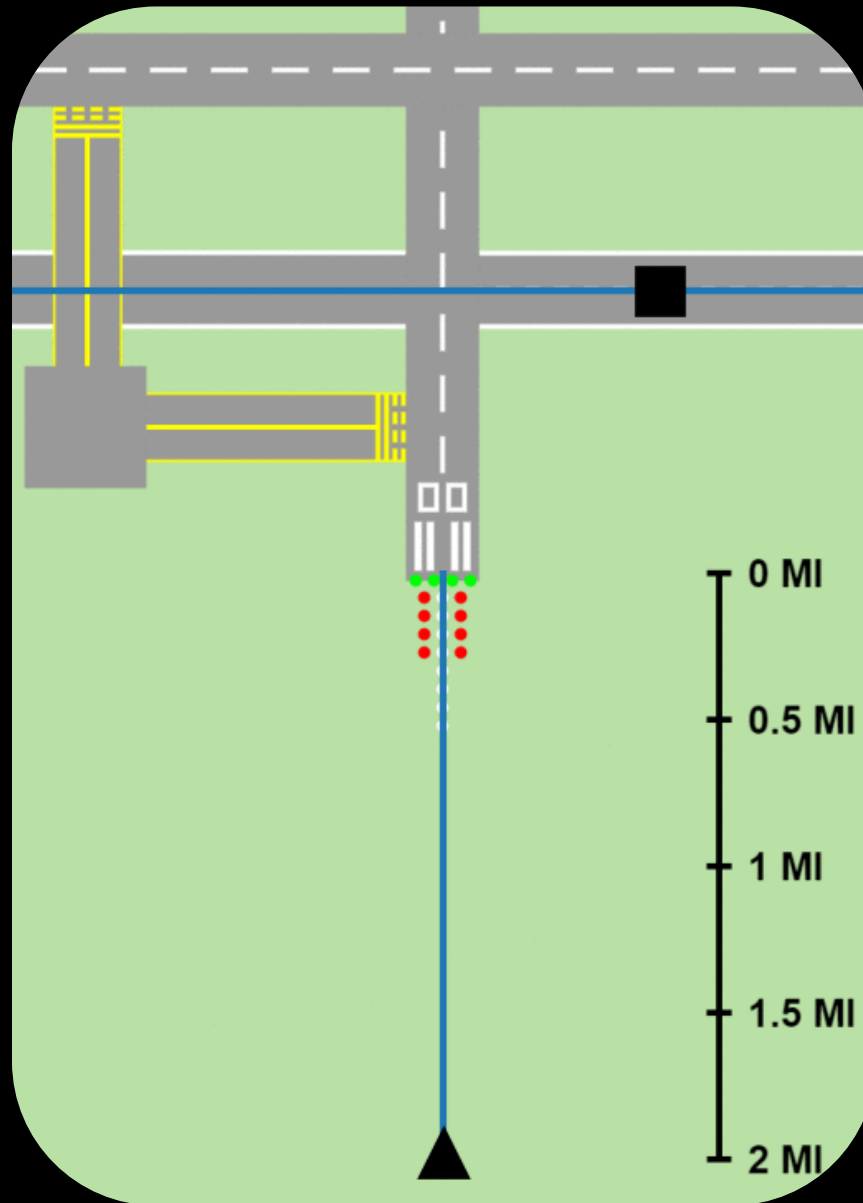
**Gauss Seidel
Value Iteration**



Policy Iteration

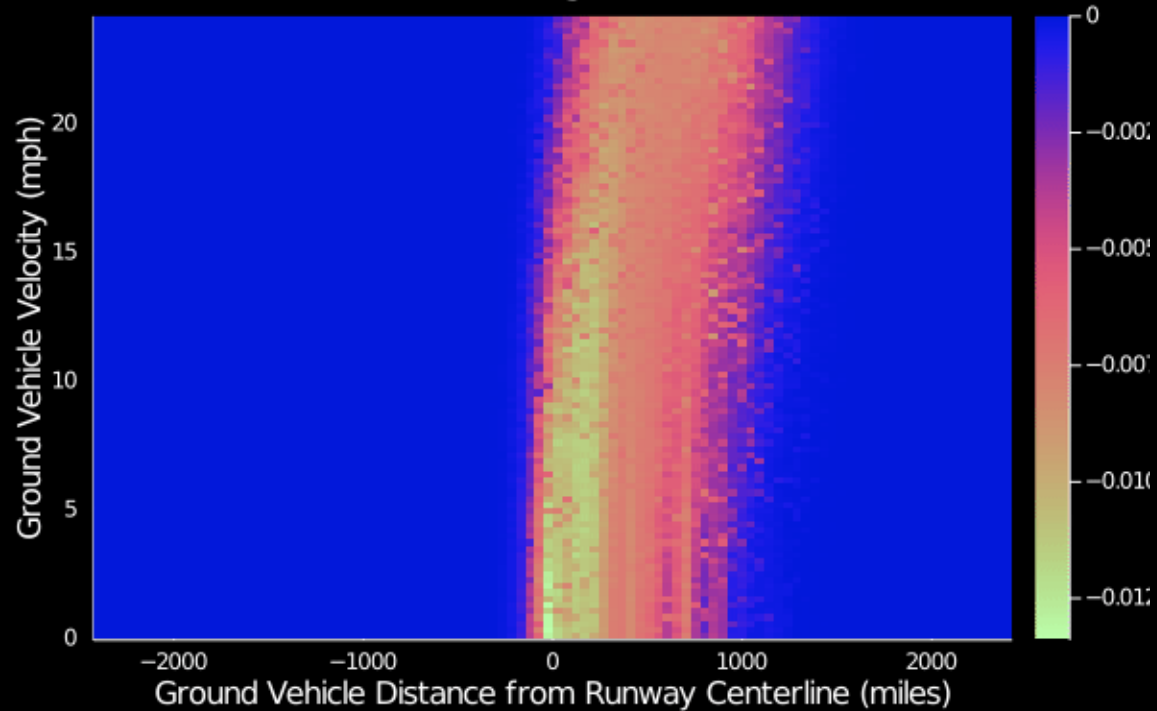
The background is a dark, monochromatic composition. It features a complex, three-dimensional wireframe mesh that appears to be draped or flowing across the frame. The mesh is composed of numerous small, interconnected triangles, creating a textured, fabric-like appearance. Overlaid on this mesh are several bright, glowing streaks of light that curve and sweep across the image, adding a sense of motion and depth. The overall aesthetic is futuristic and technical.

MDP Examples



“Passive” Policy

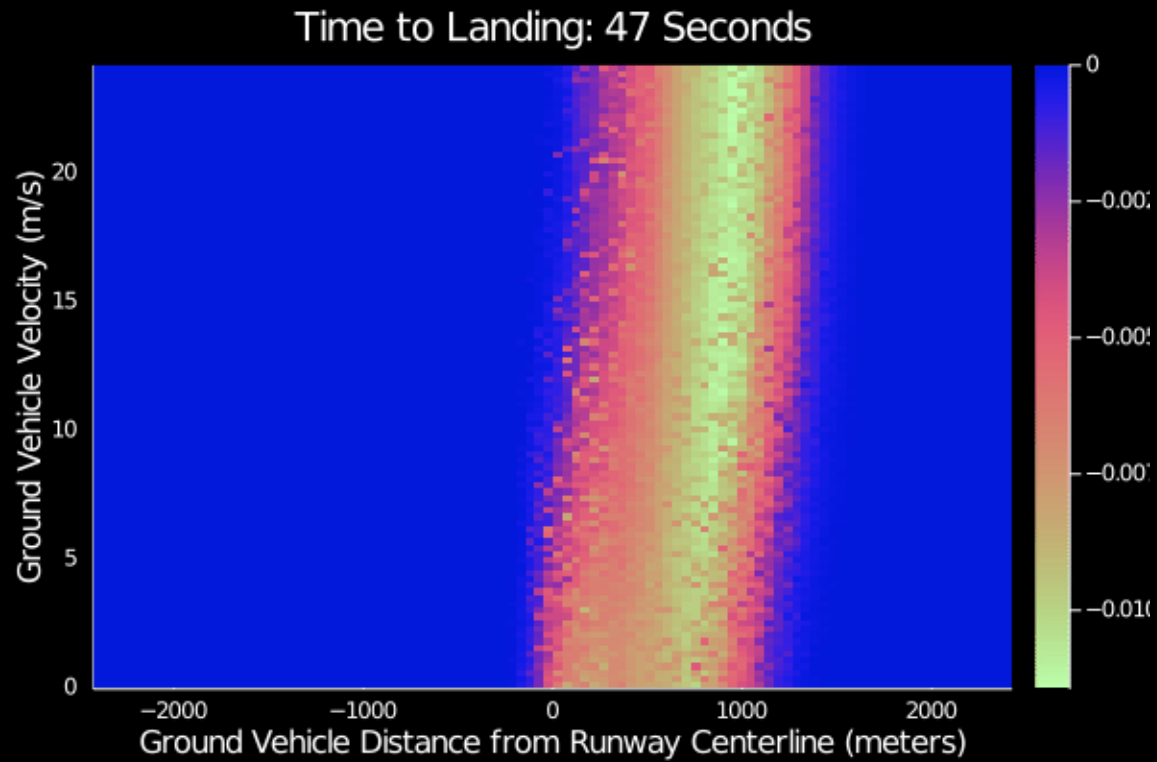
Time to Landing: 47 Seconds



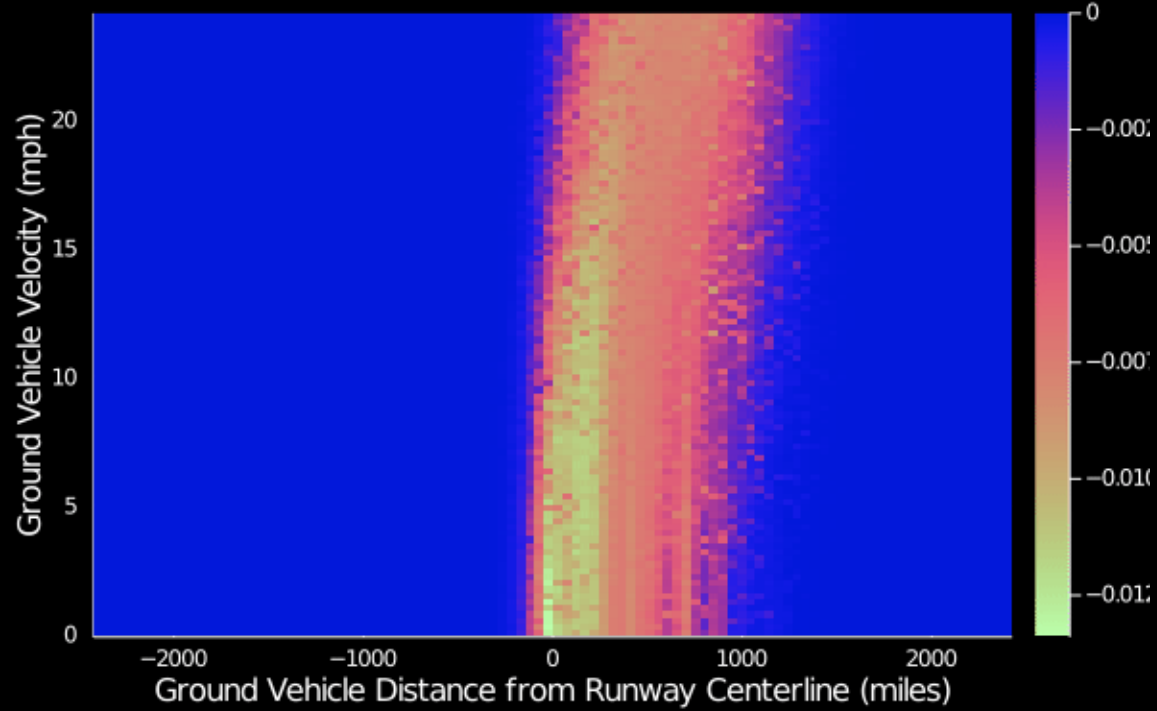
Time to Landing: 47 Seconds



“Aggressive” Policy

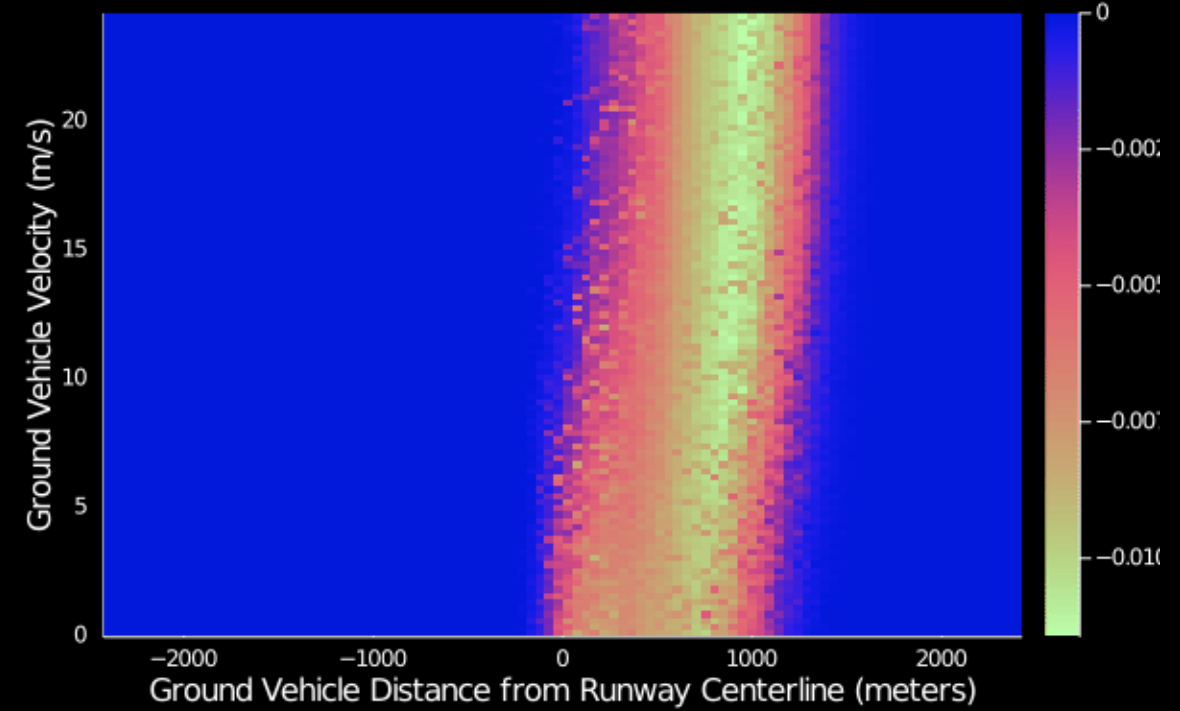


Time to Landing: 47 Seconds



Passive Intruder

Time to Landing: 47 Seconds



Aggressive Intruder

Time to Landing: 47 Seconds



Passive Intruder

Time to Landing: 47 Seconds



Aggressive Intruder

Investigation of risk-aware MDP and POMDP contingency management autonomy for UAS

Prashin Sharma *

University of Michigan, 2505 Hayward St., Ann Arbor, MI 48109

Benjamin Kraske †

University of Colorado Boulder, 3775 Discovery Dr, Boulder, CO 80303

Joseph Kim ‡

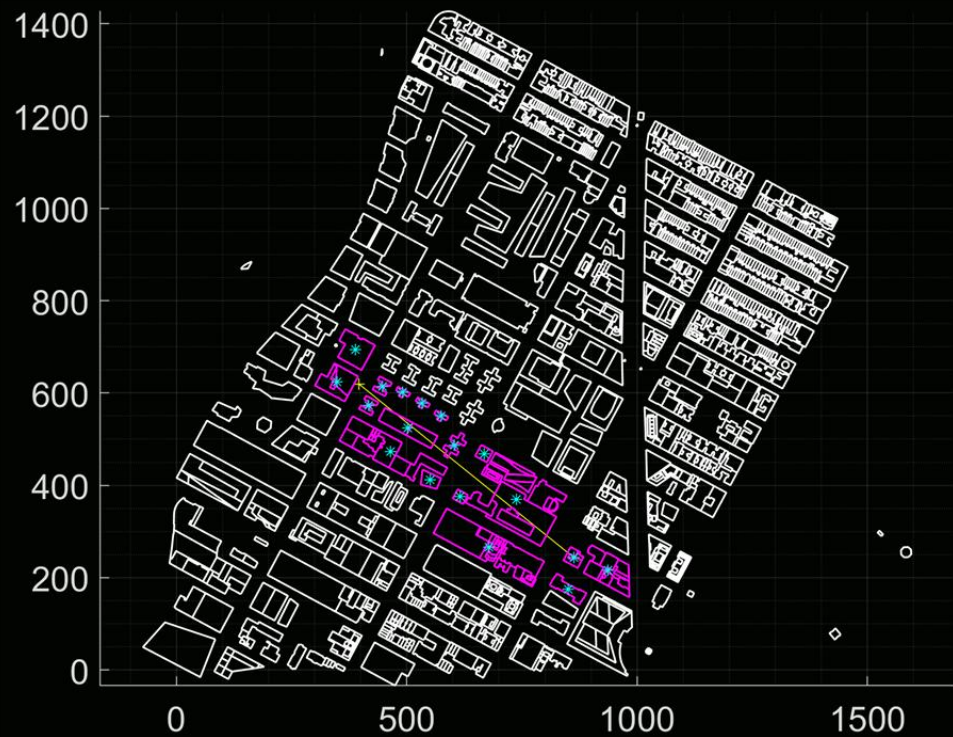
University of Michigan, 2505 Hayward St., Ann Arbor, MI 48109

Zakariya Laouar §, Zachary Sunberg ¶

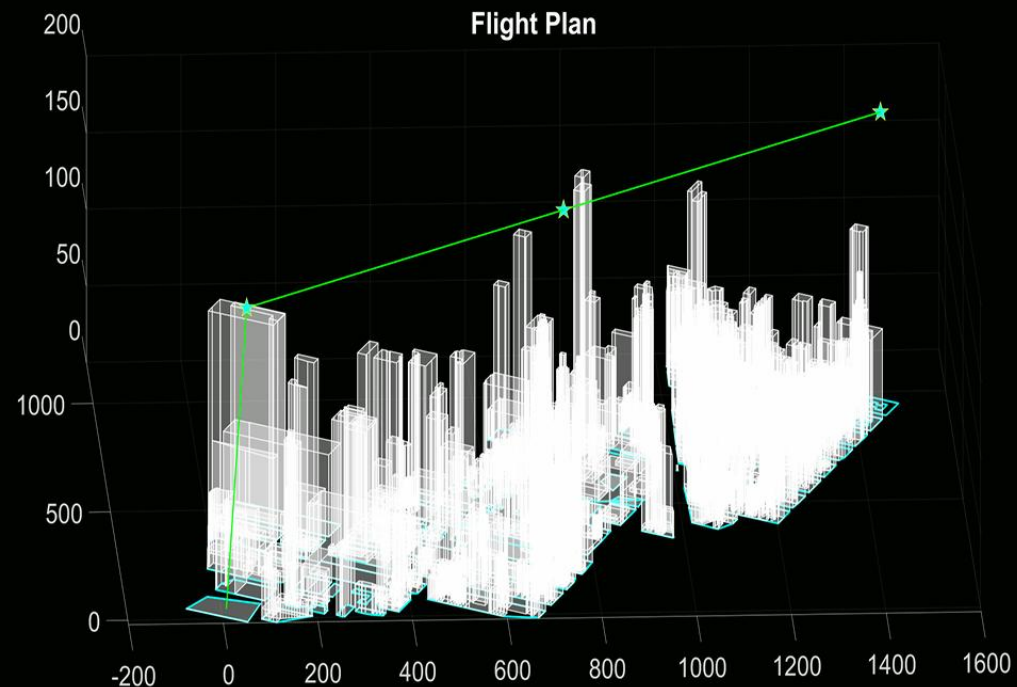
University of Colorado Boulder, 3775 Discovery Dr, Boulder, CO 80303

Ella Atkins ||

Virginia Tech, Randolph Hall, 460 Old Turner St., Blacksburg, VA 24061

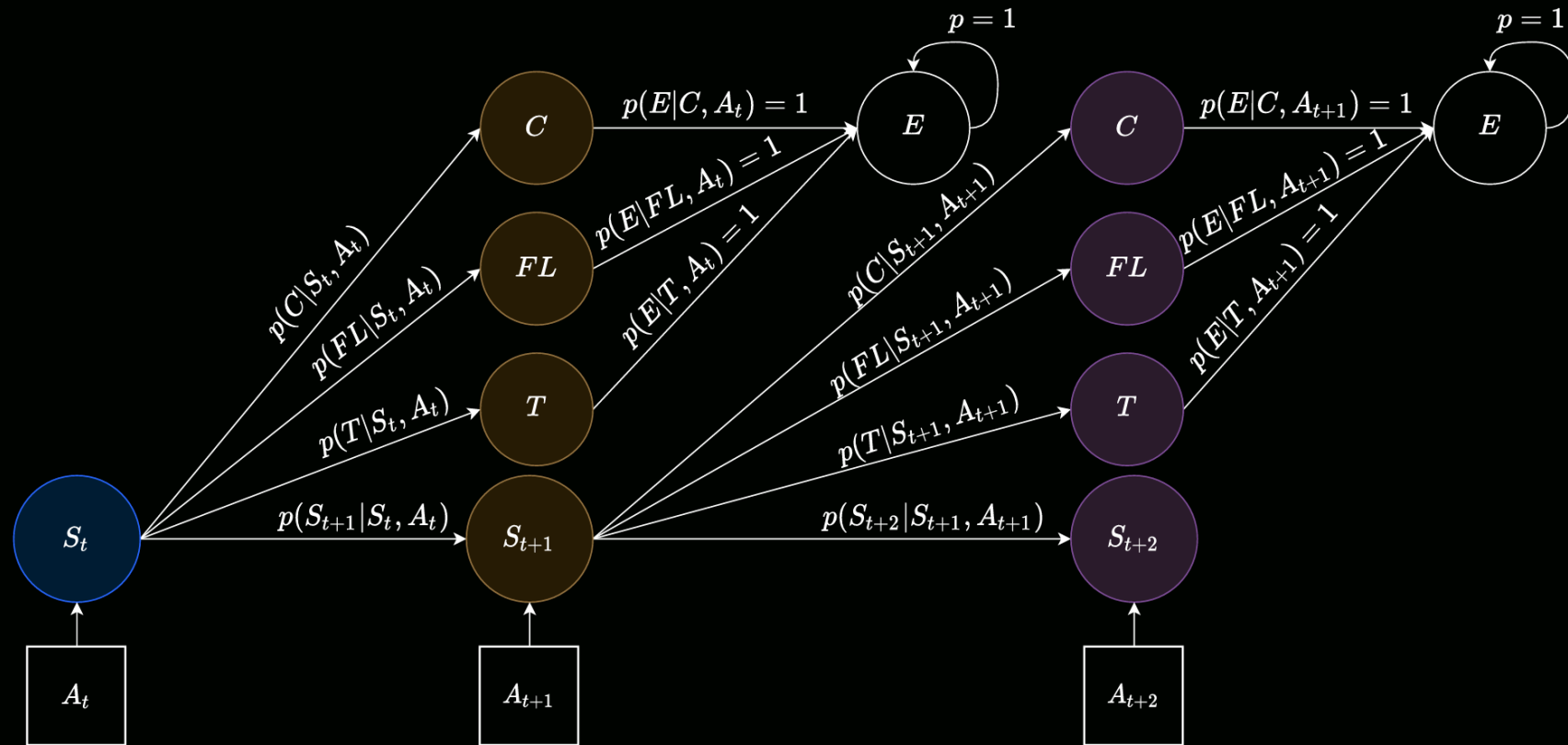


(a) Top-down view of Manhattan with nominal flight path in blue line, contingency landing sites in red stars



(b) 3-D Manhattan environment with nominal flight path in blue line and pre-planned checkpoints in Red stars.

Fig. 1 Example small UAS package delivery mission used for MDP and POMDP CMA simulations.



State				Action
C: Complete	Flight Status (FS)	Motor Health (MH)	Battery Health (BH)	NoOP Terminate Land ASAP Land Pract
T: Terminate	N: Nominal	NF : No Fault	G : Good	
FL: Failure	ELASAP : Emergency Landing ASAP	SF : Spalling Fault	M : Medium	
E: End	ELPract : Emergency Landing Practical	JF: Jam Fault	P : Poor	
		Motor Margin (MM)	Reachability Margin (RM)	
		0 : MM < 0	0 : RM < 0	
		1: 0 <= MM < 1	1: 0 <= RM < 1	

Optimizing the simultaneous management of blood pressure and cholesterol for type 2 diabetes patients



J.E. Mason^{a,*}, B.T. Denton^{a,1}, N.D. Shah^b, S.A. Smith^c

^aEdward P. Fitts Department of Industrial & Systems Engineering, North Carolina State University, 111 Lampe Drive, Raleigh, NC 27613, USA

^bDivision of Health Care Policy & Research, Mayo Clinic College of Medicine, 200 First Street SW, Rochester, MN 55905, USA

^cDivision of Endocrinology, Diabetes, Nutrition, & Metabolism, Mayo Clinic College of Medicine, 200 First Street SW, Rochester, MN 55905, USA

ARTICLE INFO

Article history:

Received 16 November 2012

Accepted 14 September 2013

Available online 25 September 2013

Keywords:

OR in medicine

Markov decision process

Type 2 diabetes

Blood pressure treatment

Cholesterol treatment

ABSTRACT

We present a Markov decision process (MDP) model to determine the optimal timing of blood pressure and cholesterol medications. We study the use of our model for a high-risk population of patients with type 2 diabetes; however, the model and methods we present are applicable to the general population. We compare the optimal policies based on our MDP to published guidelines for initiation of blood pressure and cholesterol medications over the course of a patient's lifetime. We also present a bicriteria analysis that illustrates the trade off between quality-adjusted life years and costs of treatment.

© 2013 Elsevier B.V. All rights reserved.

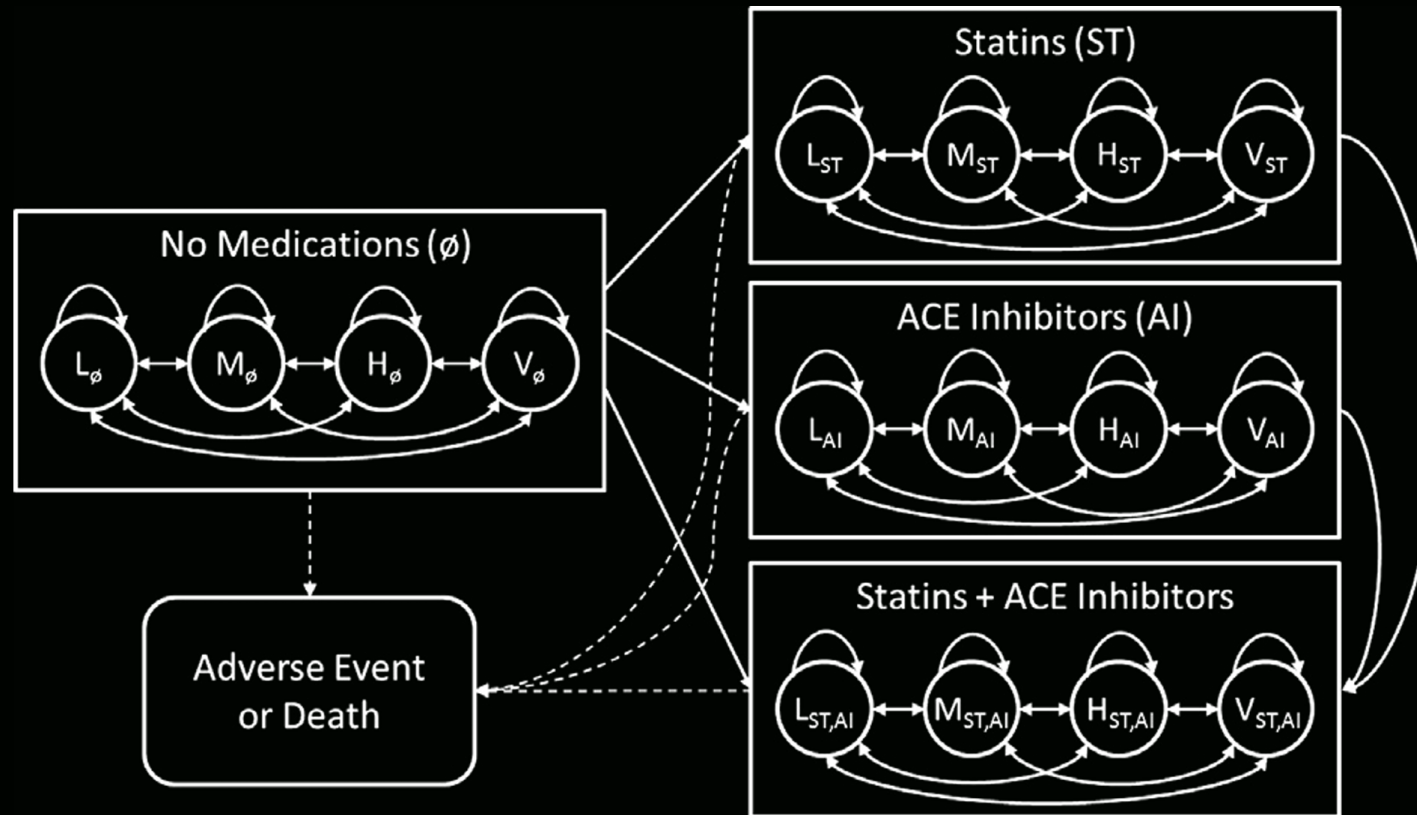
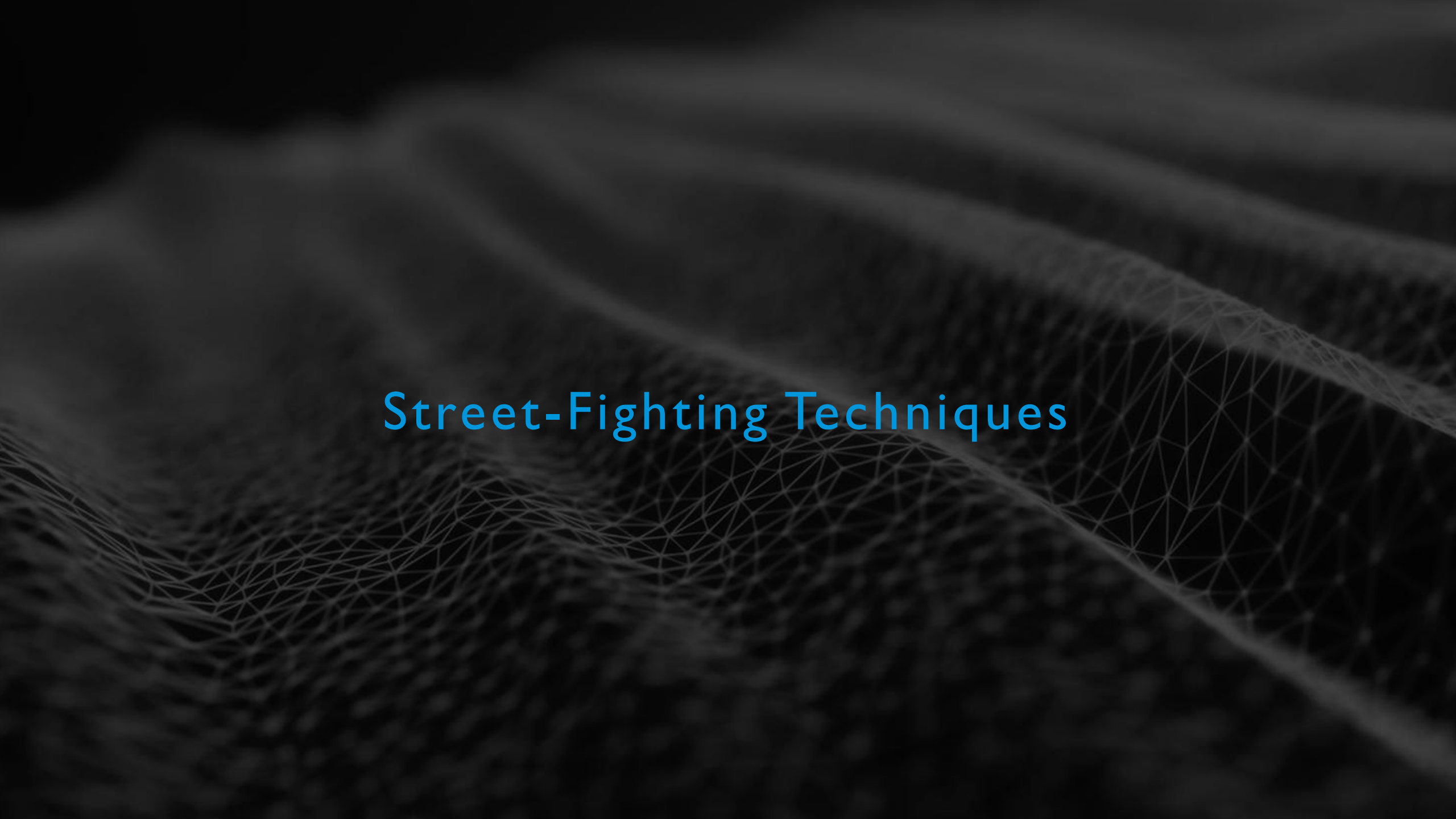
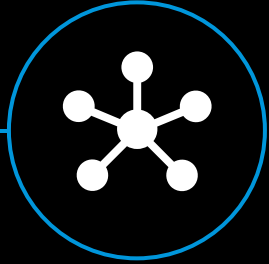


Fig. 1. Simplified state transition diagram for the case of two medications. When medications are initiated (actions denoted by the solid lines), the risk factors are improved and the probability of the occurrence of an adverse event (denoted by the dashed lines) is reduced.

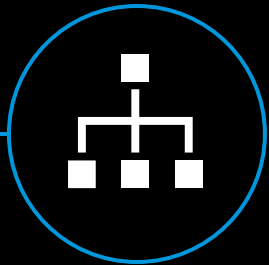
The background is a dark, monochromatic abstract composition. It features a complex, three-dimensional wireframe mesh that appears to be draped or flowing across the frame. The mesh is composed of numerous small, interconnected triangles, creating a textured, fabric-like appearance. Overlaid on this mesh are several broad, sweeping, and slightly blurred lines that suggest movement and depth. The overall effect is a sense of dynamic, organic structure.

Street-Fighting Techniques



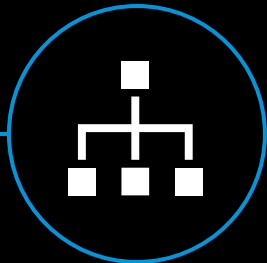
Value Function Approximation

Use parametric approximations, kernel smoothing, or neural networks



Monte Carlo Tree Search

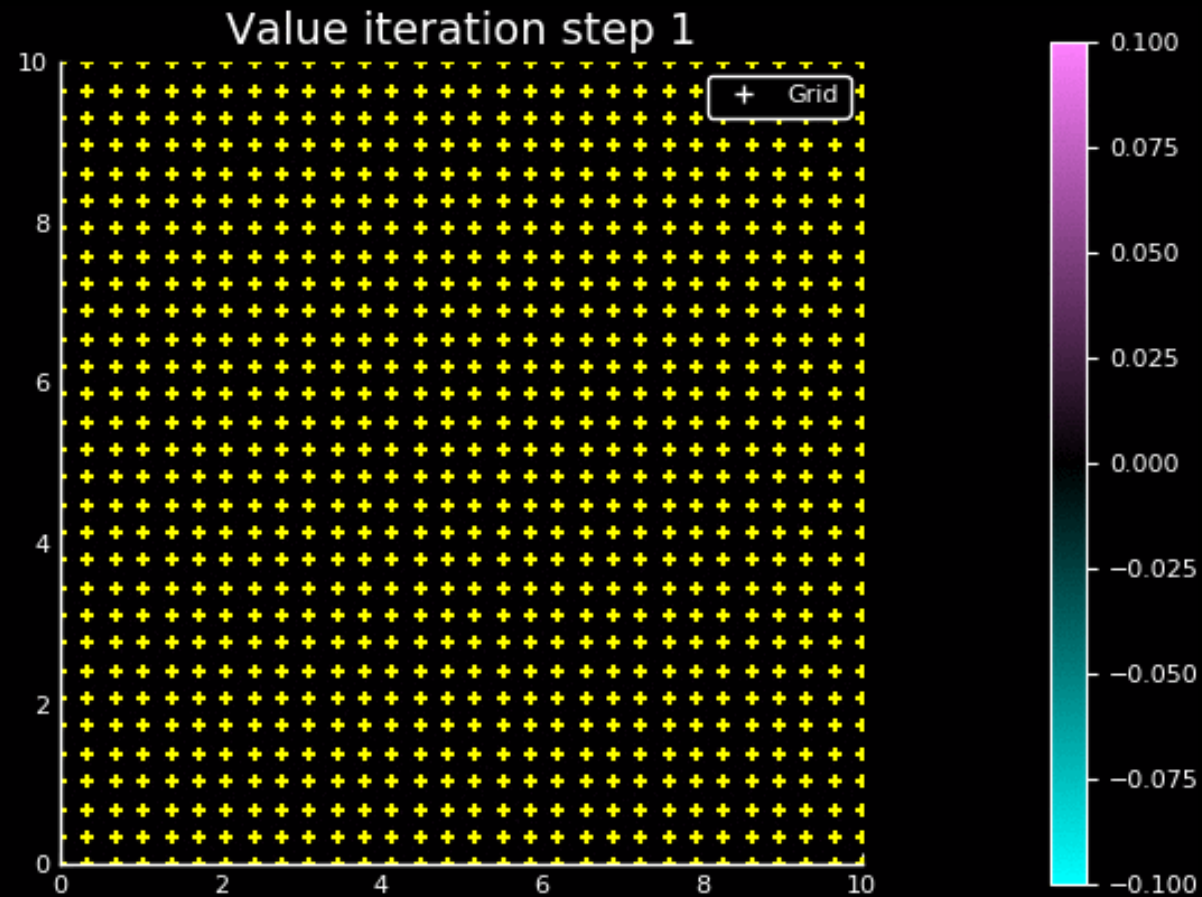
Workhorse solver for online planning



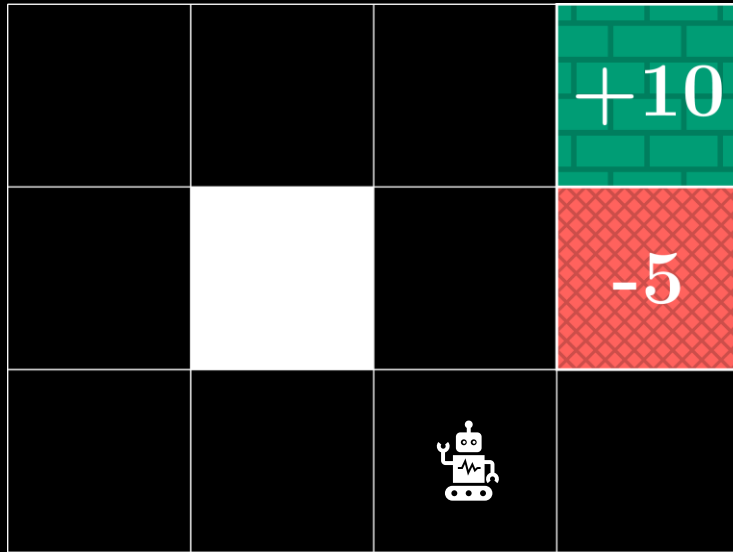
Progressive Widening

Heuristic approach for MCTS with continuous state/action spaces

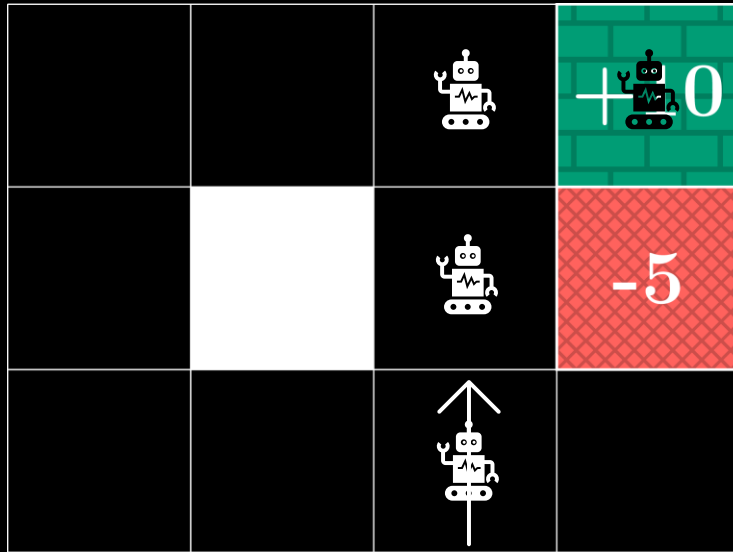
Local Approximation Value Iteration



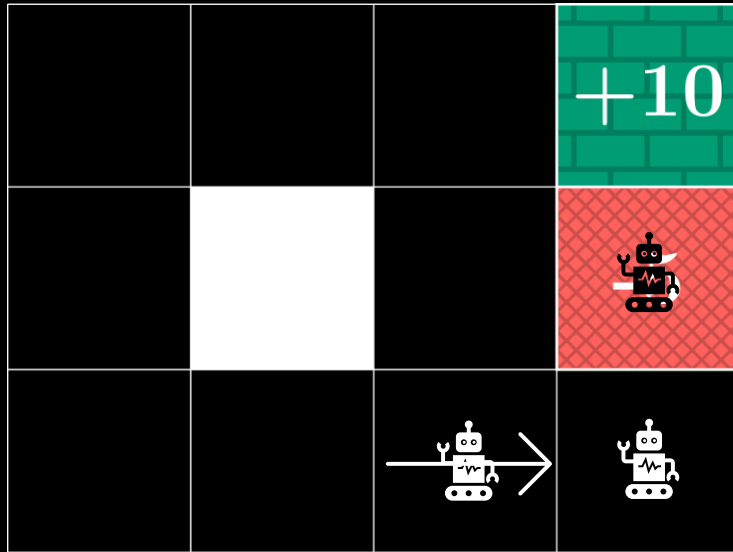
Monte Carlo Tree Search



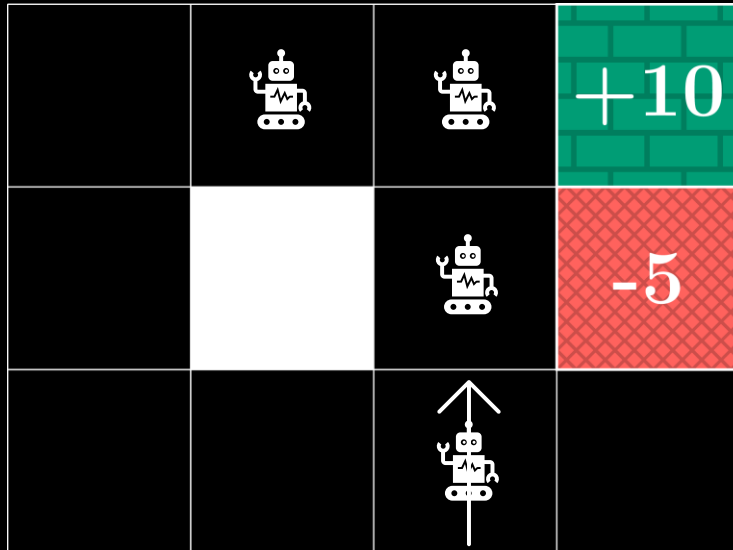
Monte Carlo Tree Search



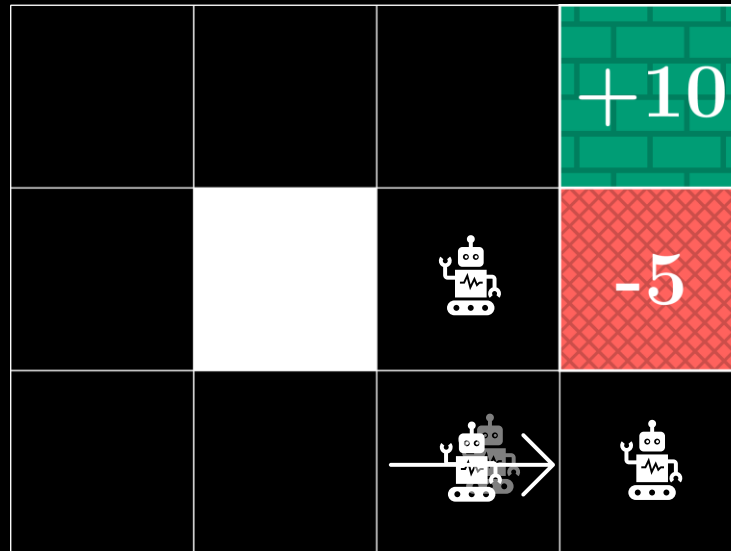
Monte Carlo Tree Search



Monte Carlo Tree Search



Monte Carlo Tree Search



Monte Carlo Tree Search

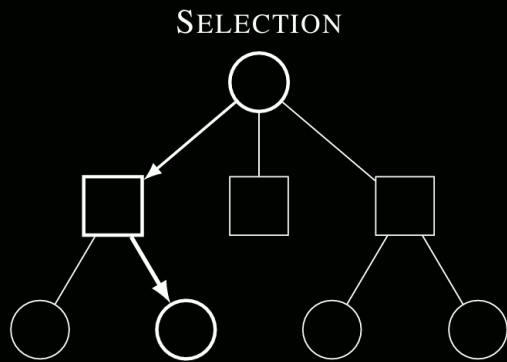
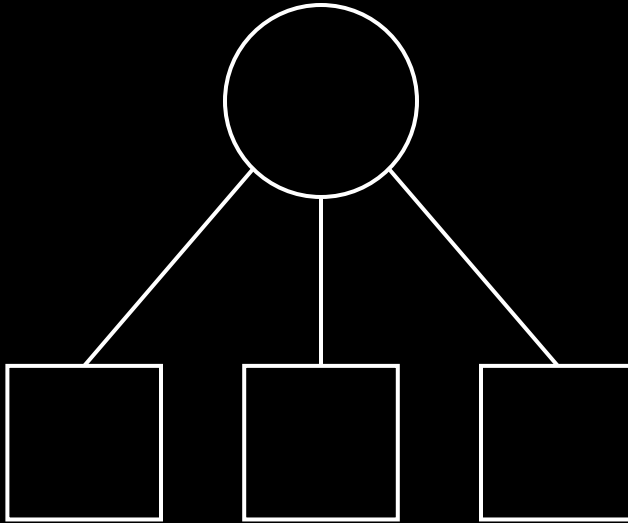
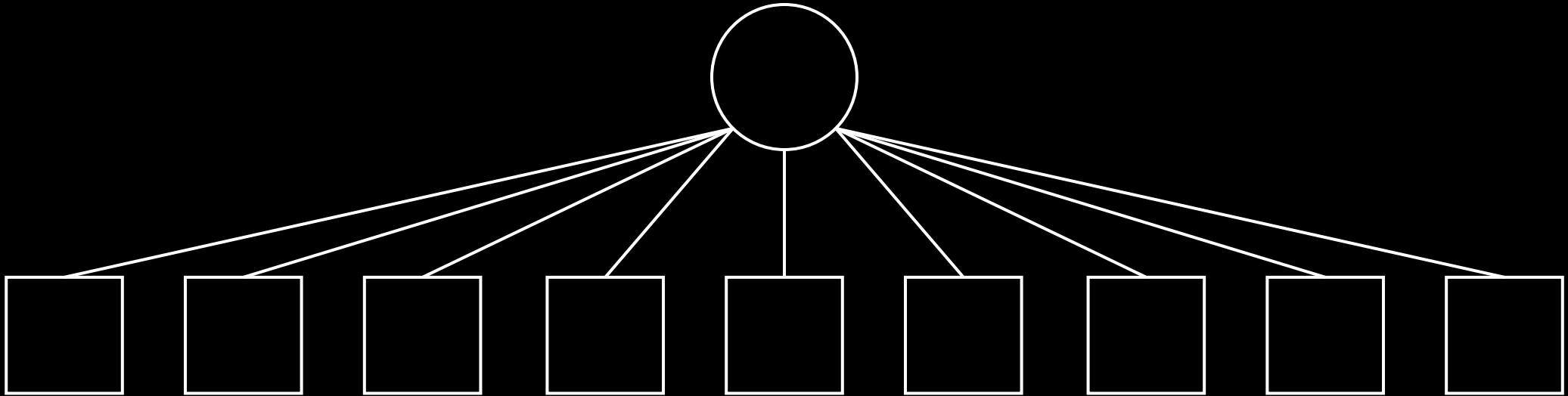


Image credit: [2011.02559] Adaptive Stress Testing of Trajectory Predictions in Flight Management Systems
By Moss et al.

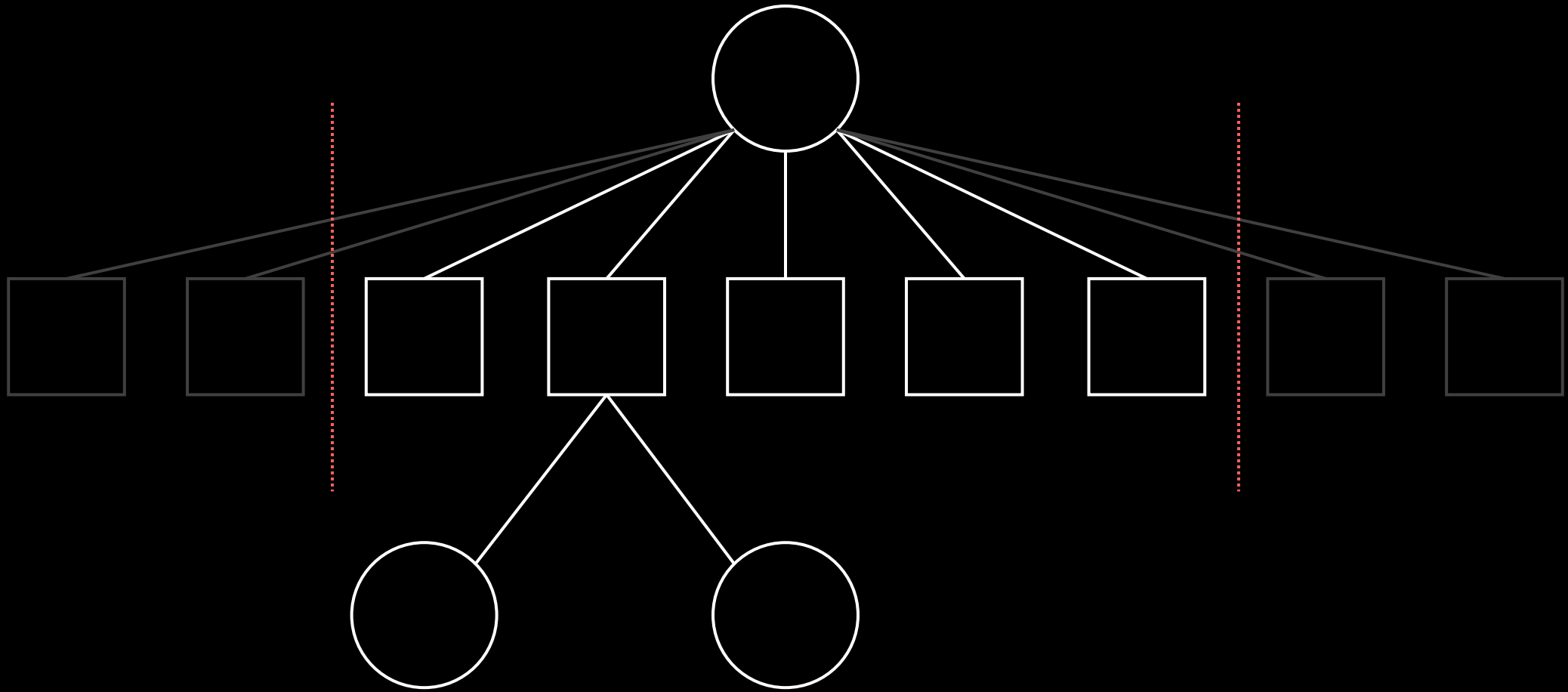
Progressive Widening

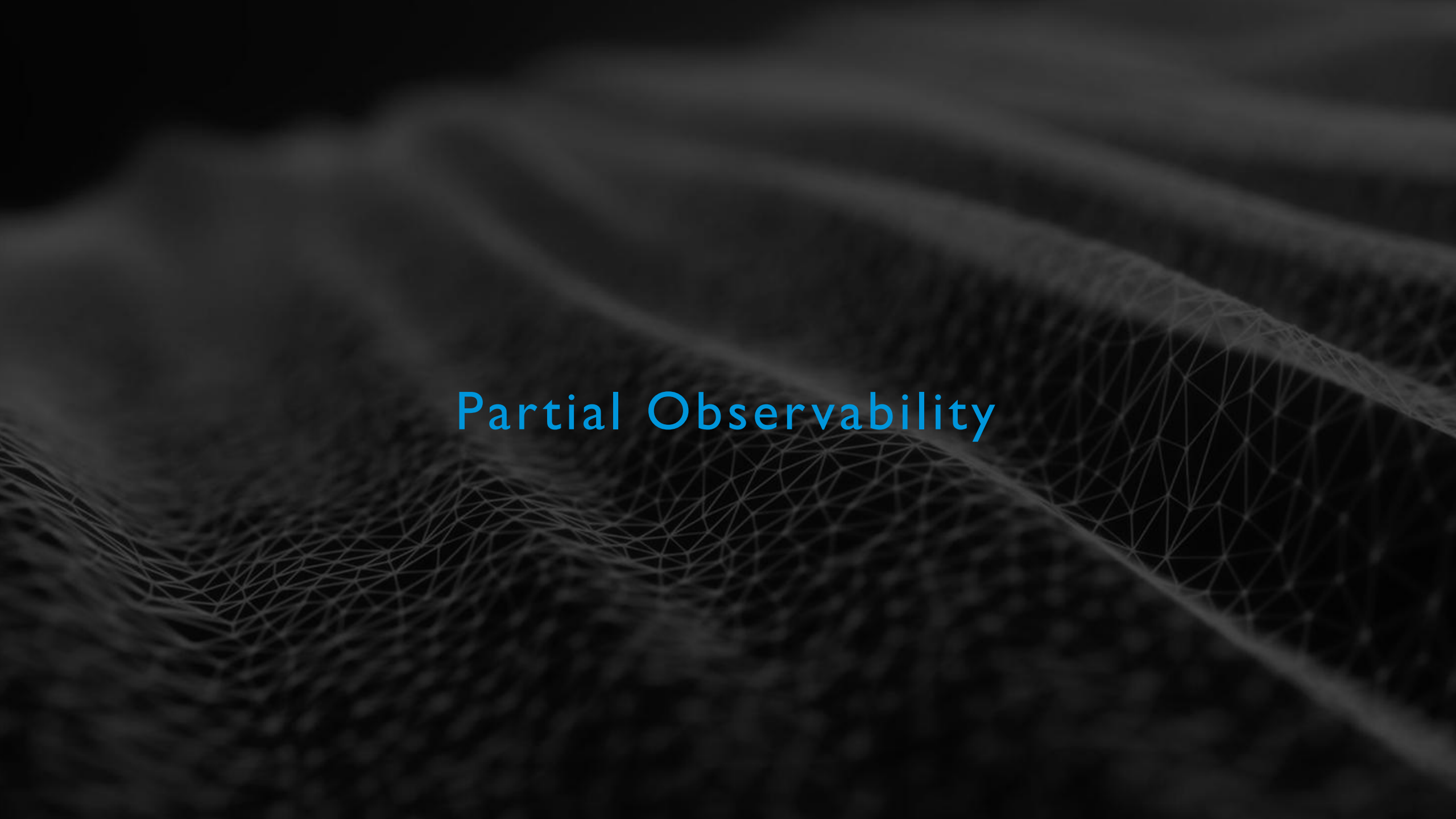


Progressive Widening



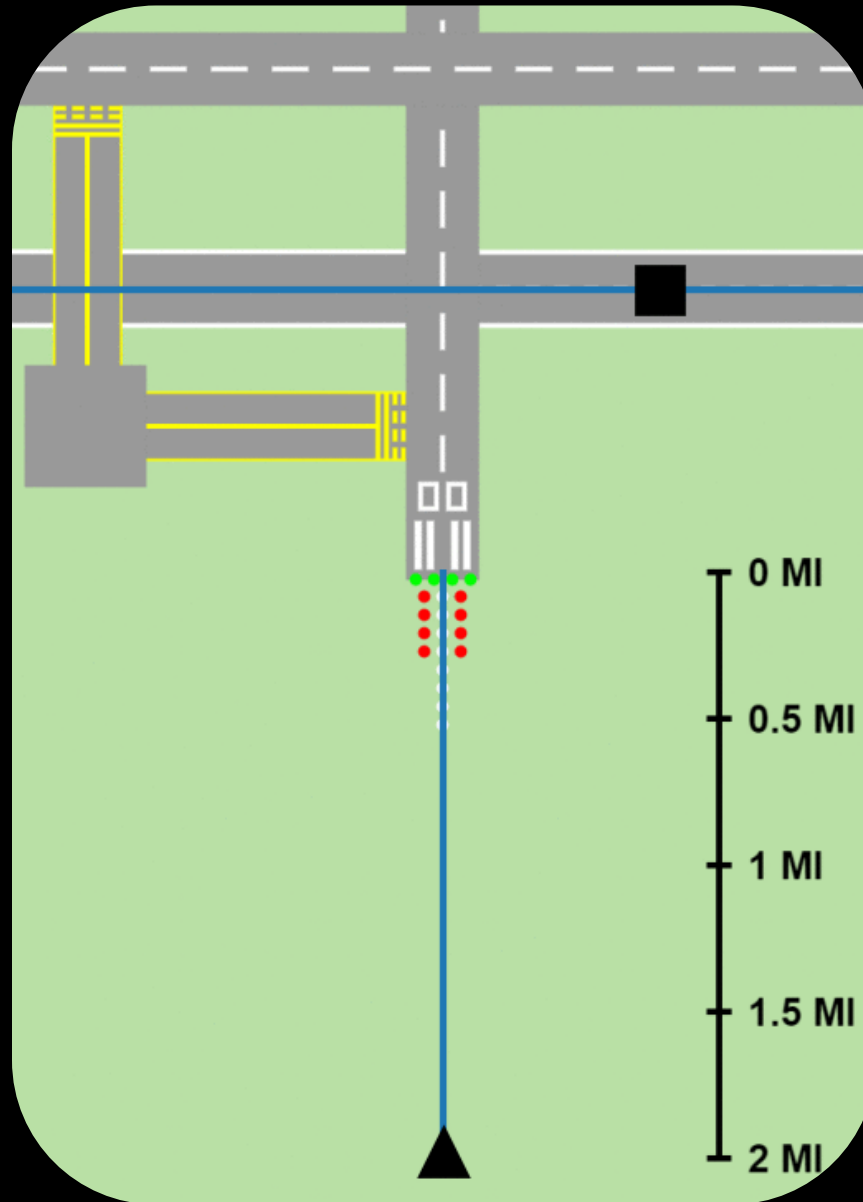
Progressive Widening

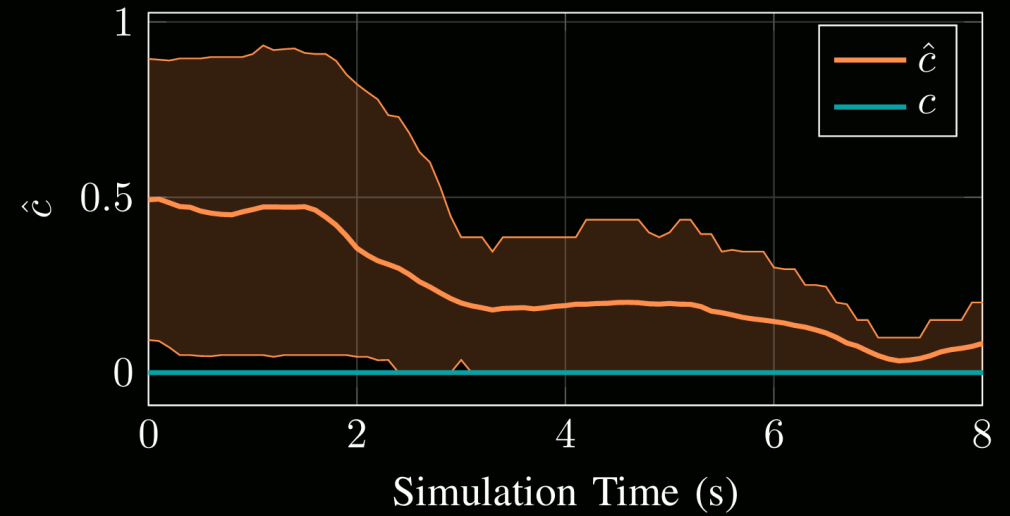
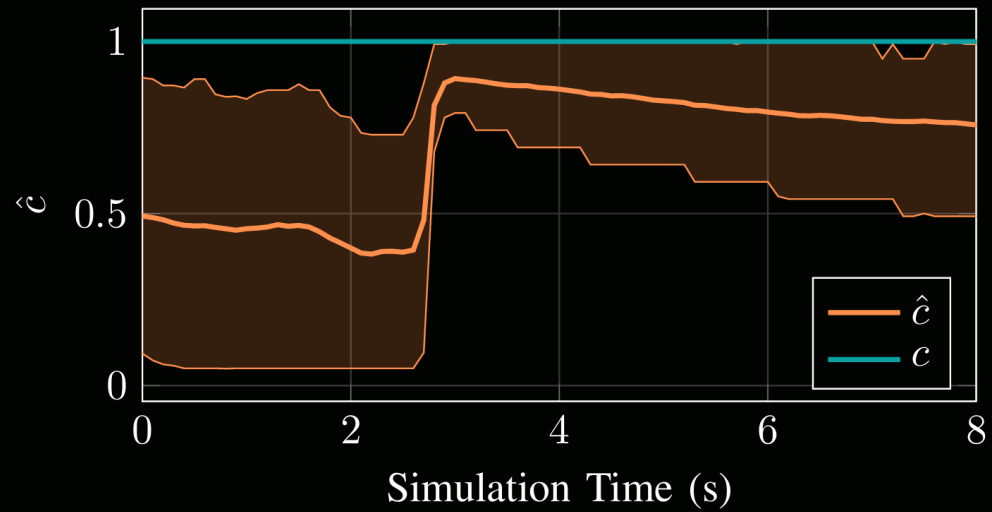
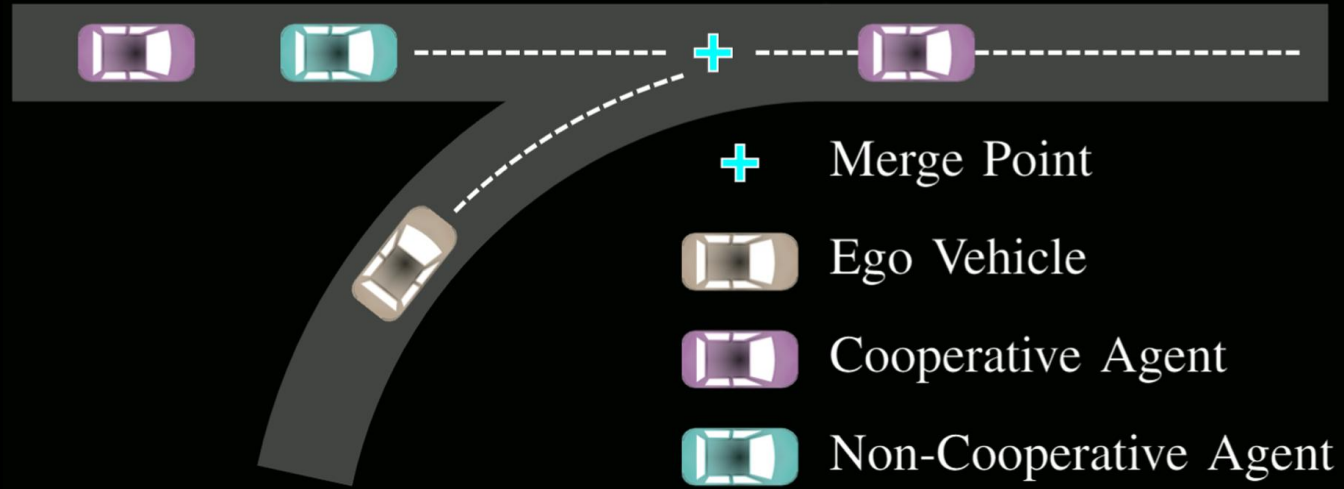




Partial Observability







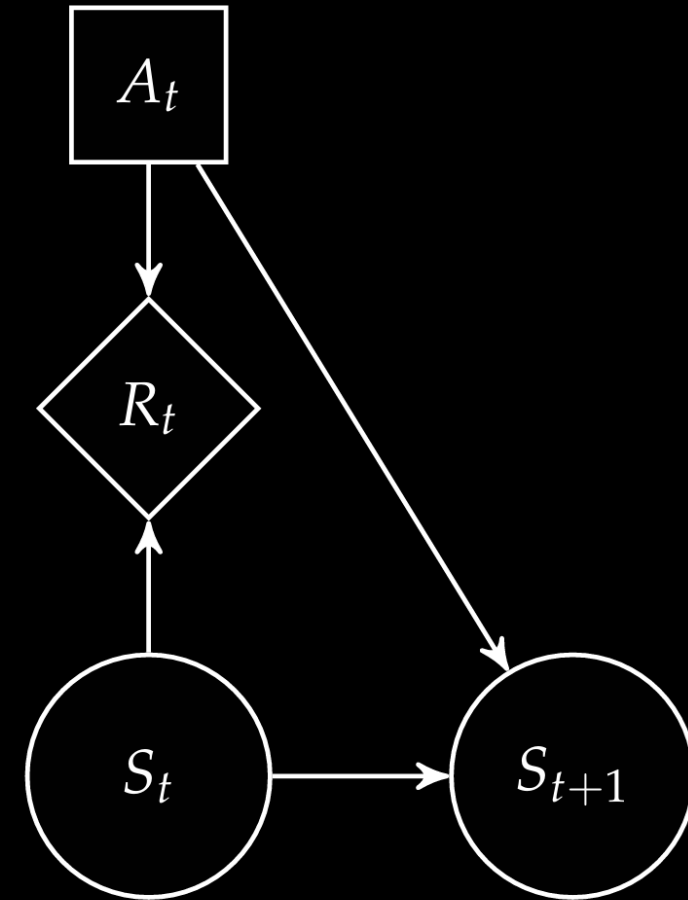
Markov Decision Process

State Space $\mathcal{S}$

Action Space $\mathcal{A}$

Transition Model $T(s' | s, a)$

Reward Function $R(s, a)$



Partially Observable MDP

State Space $\mathcal{S}$

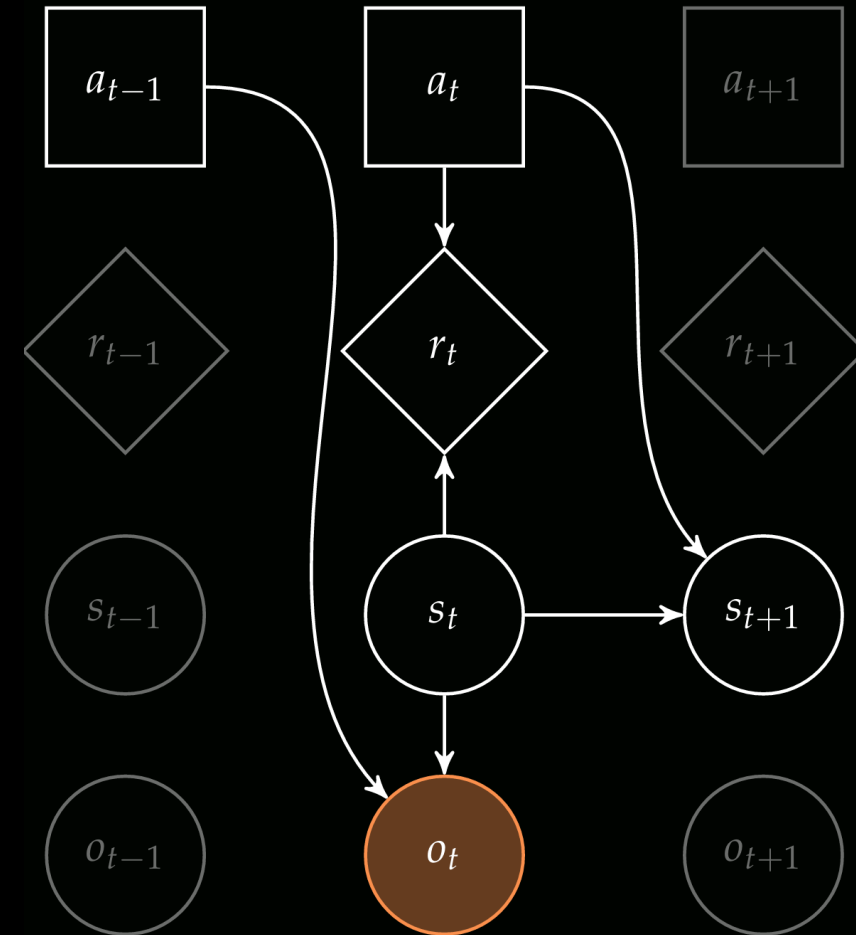
Action Space $\mathcal{A}$

Transition Model $T(s' | s, a)$

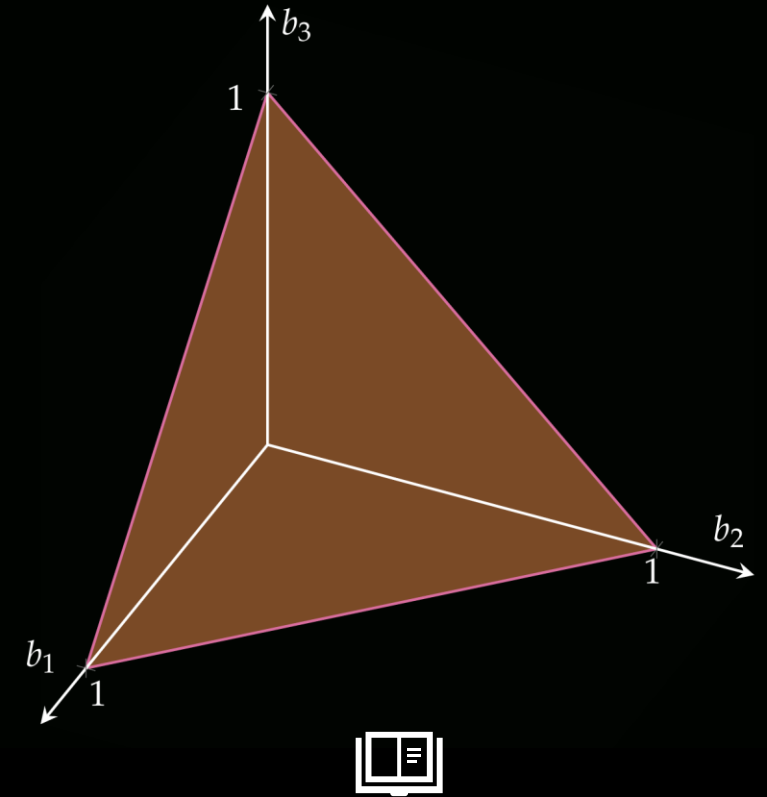
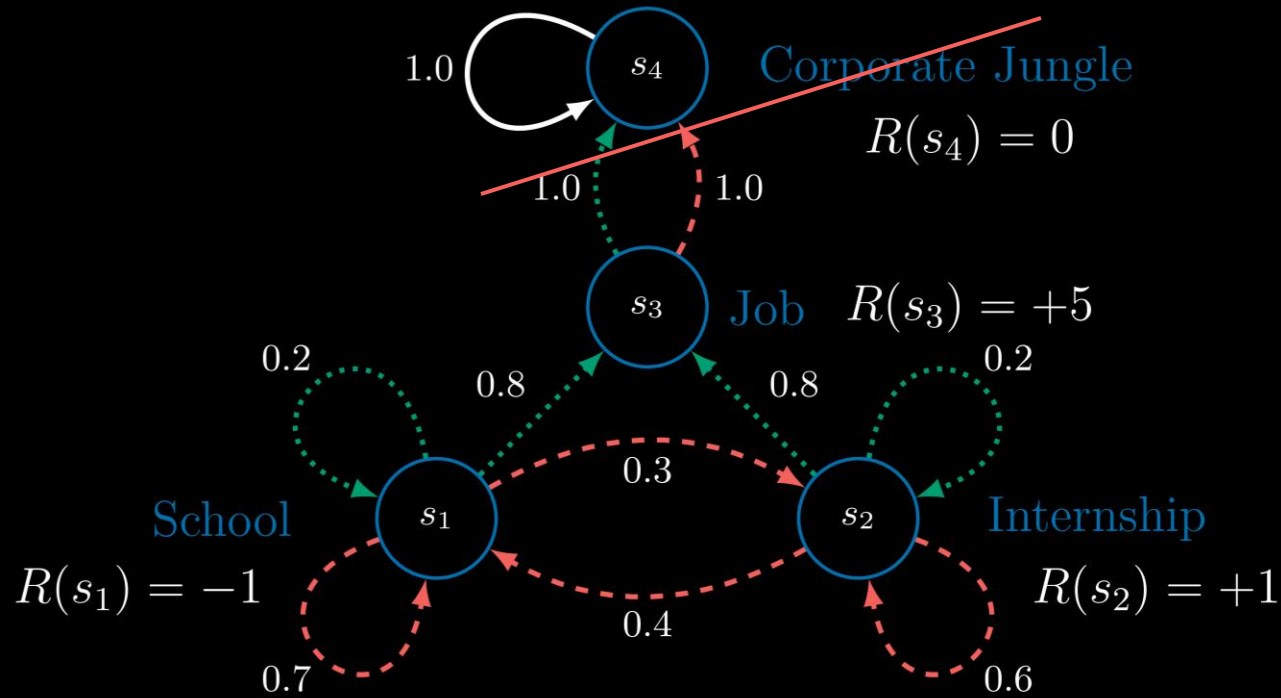
Reward Function $R(s, a)$

Observation Space $\mathcal{O}$

Observation Function $O(o | s', a)$

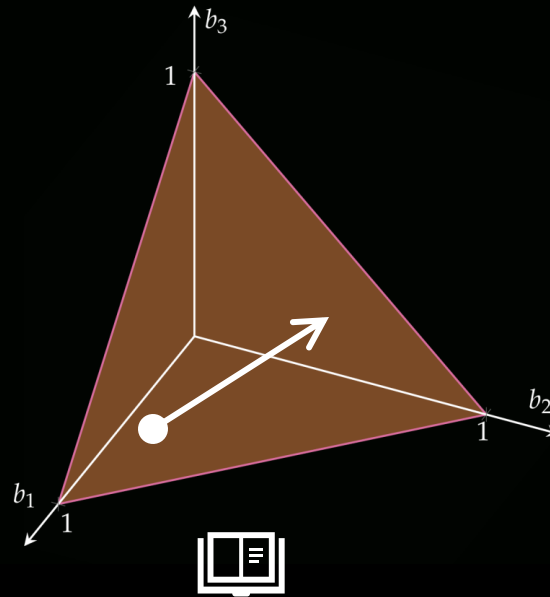


Beliefs over States



Discrete Belief Update

$$b'(s') = O(o \mid a, s') \sum_s T(s' \mid s, a) b(s)$$



Exercise 19.10. Suppose we are performing in-flight monitoring of an aircraft. The aircraft is either in a state of normal operation s^0 or a state of malfunction s^1 . We receive observations through the absence of a warning w^0 or the presence of a warning w^1 . We can choose to allow the plane to continue to fly m^0 or send the plane in for maintenance m^1 . We have the following transition and observation dynamics, where we assume that the warnings are independent of the actions, given the status of the plane:

$$T(s^0 \mid s^0, m^0) = 0.95$$

$$O(w^0 \mid s^0) = 0.99$$

$$T(s^0 \mid s^0, m^1) = 1$$

$$O(w^1 \mid s^1) = 0.7$$

$$T(s^1 \mid s^1, m^0) = 1$$

$$T(s^0 \mid s^1, m^1) = 0.98$$

Given the initial belief $\mathbf{b} = [0.95, 0.05]$, compute the updated belief $\mathbf{b}'$, given that we allow the plane to continue to fly and we observe a warning.



$$T(s^0 \mid s^0, m^0) = 0.95$$

$$O(w^0 \mid s^0) = 0.99$$

$$T(s^0 \mid s^0, m^1) = 1$$

$$O(w^1 \mid s^1) = 0.7$$

$$T(s^1 \mid s^1, m^0) = 1$$

$$T(s^0 \mid s^1, m^1) = 0.98$$

Given the initial belief $\mathbf{b} = [0.95, 0.05]$, compute the updated belief $\mathbf{b}'$, given that we allow the plane to continue to fly and we observe a warning.

$$b'(s^0) \propto O(w^1 \mid s^0) \sum_s T(s^0 \mid s, m^0) b(s)$$



$$T(s^0 \mid s^0, m^0) = 0.95$$

$$O(w^0 \mid s^0) = 0.99$$

$$T(s^0 \mid s^0, m^1) = 1$$

$$O(w^1 \mid s^1) = 0.7$$

$$T(s^1 \mid s^1, m^0) = 1$$

$$T(s^0 \mid s^1, m^1) = 0.98$$

Given the initial belief $\mathbf{b} = [0.95, 0.05]$, compute the updated belief $\mathbf{b}'$, given that we allow the plane to continue to fly and we observe a warning.

$$b'(s^1) \propto O(w^1 \mid s^1) \sum_s T(s^1 \mid s, m^0) b(s)$$



$$b'(s^0) = \frac{b'(s^0)}{b'(s^0) + b'(s^1)} \approx 0.117$$

$$b'(s^1) = \frac{b'(s^1)}{b'(s^0) + b'(s^1)} \approx 0.883$$

$$b' \approx [0.117, 0.883]$$





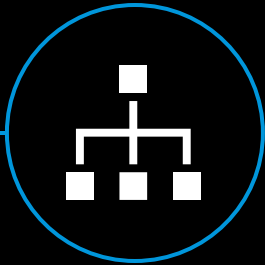
69



A *belief* is a probability distribution over states



All POMDP solvers require *belief updates*

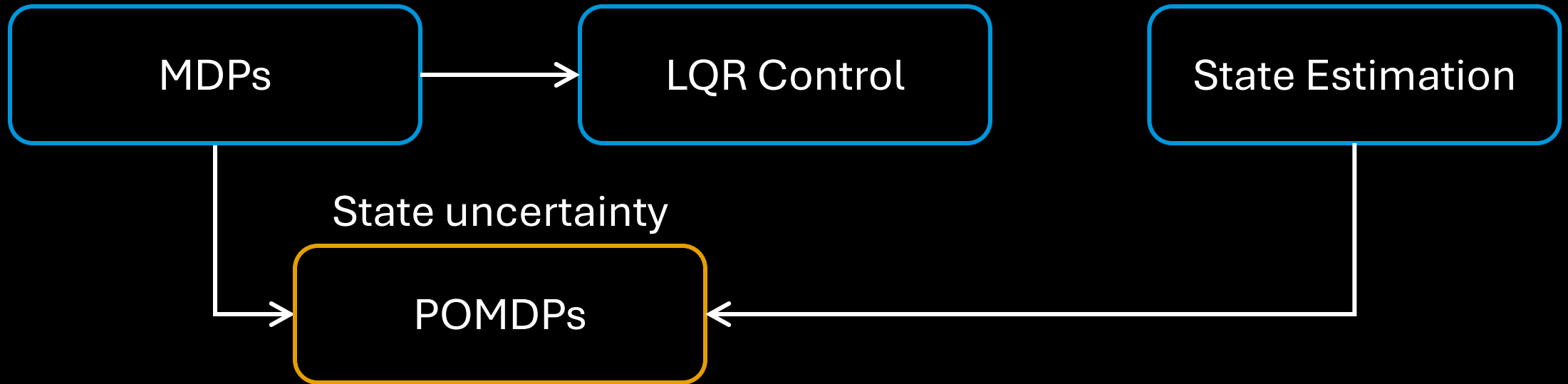


Solutions are almost always approximate

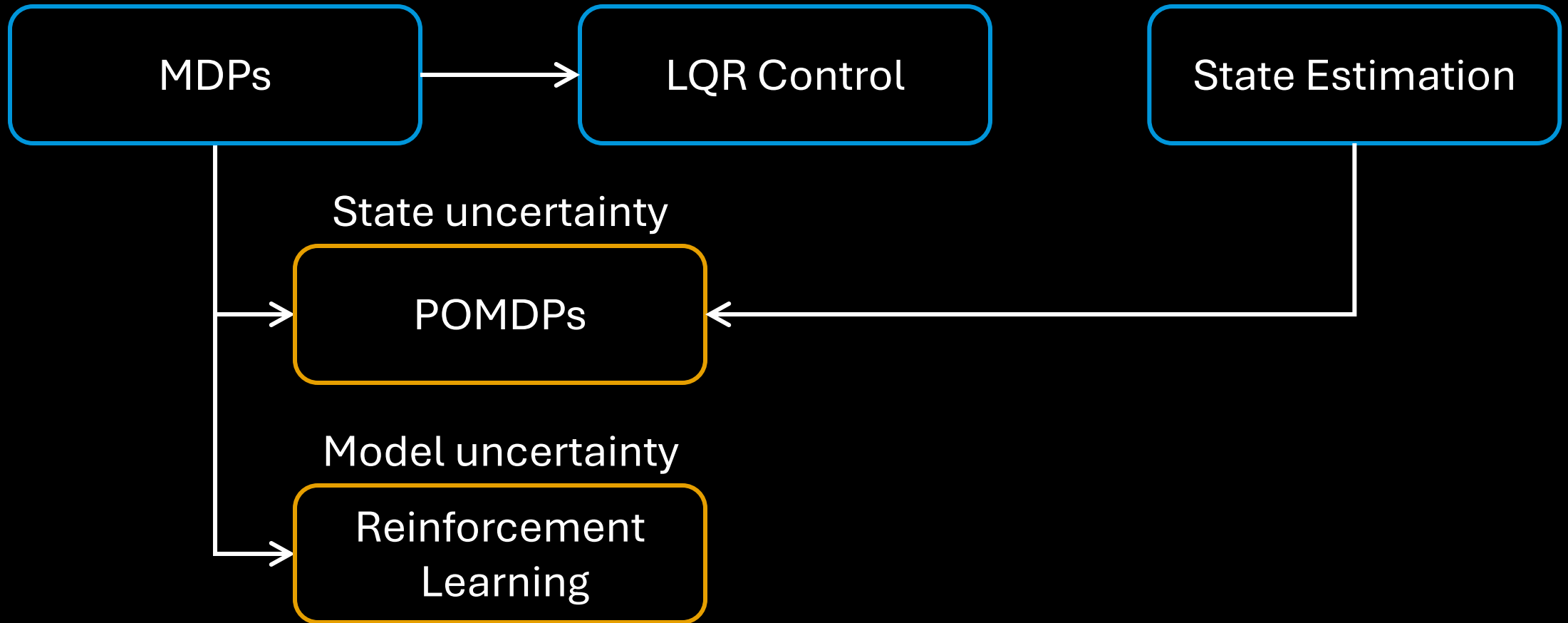
Big Picture

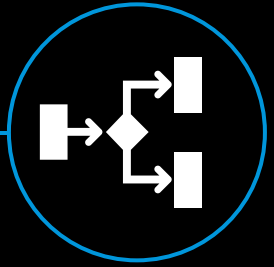


Big Picture

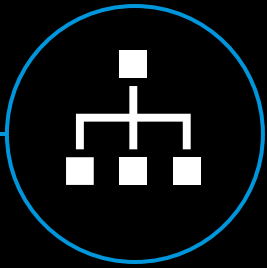


Big Picture





MDPs are best suited for high-level decisions



When in doubt, try a tree search solver



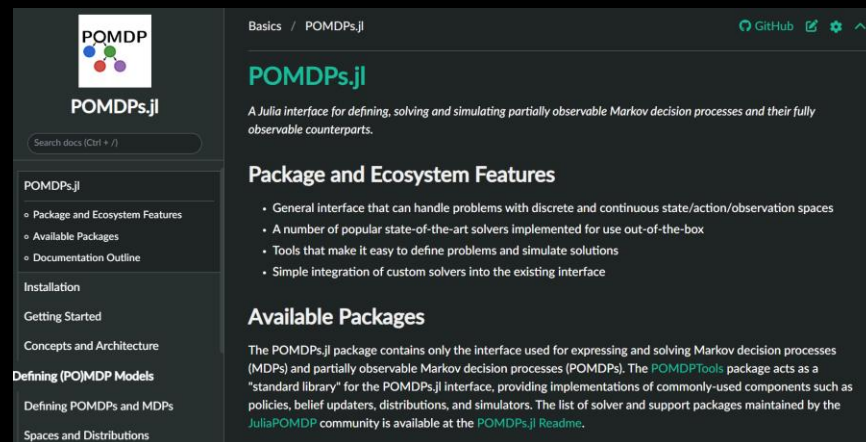
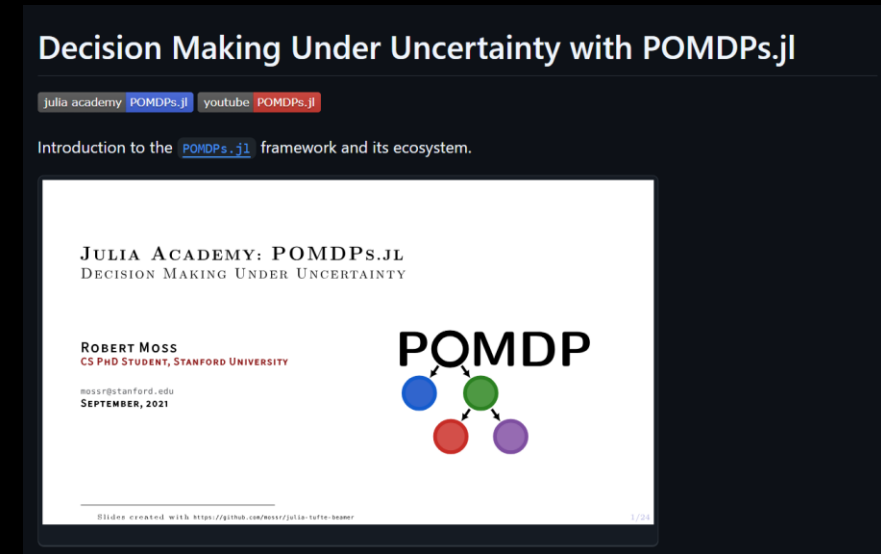
Domain knowledge is your best friend



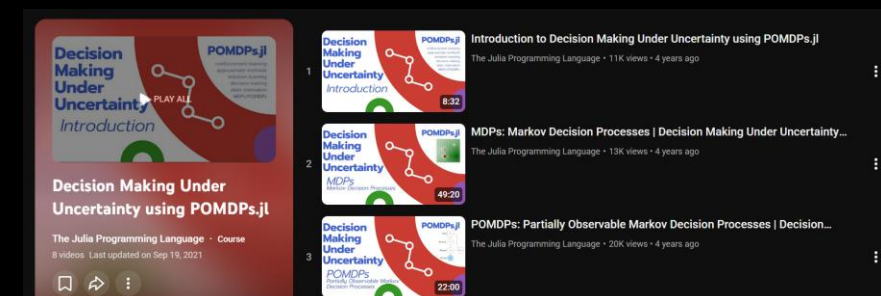
Resources



Algorithms for Decision Making



JuliaPOMDP/POMDPs.jl



JuliaAcademy/Decision-Making-Under-Uncertainty:
Decision making under uncertainty using the POMDPs.jl
ecosystem taught by Robert Moss



Thank you

Questions?